TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A MAJOR PROJECT REPORT

ON

CIRCUITRON: CIRCUIT RECOGNITION, TRANSFORMATION, AND ANALYSIS

Submitted By:

Anveshan Timsina	(THA078BEI005)
Sandesh Dhital	(THA078BEI034)
Saroj Nagarkoti	(THA078BEI039)
Utsab Dahal	(THA078BEI046)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

March 2026

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A MAJOR PROJECT REPORT

ON

CIRCUITRON: CIRCUIT RECOGNITION, TRANSFORMATION, AND ANALYSIS

Submitted By:

Anveshan Timsina (THA078BEI005)

Sandesh Dhital (THA078BEI034)

Saroj Nagarkoti (THA078BEI039)

Utsab Dahal (THA078BEI046)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

In partial fulfillment for the award of the

Bachelors Degree in Electronics, Communication and Information Engineering

Under the Supervision of

Er. Kobid Karkee

March 2026

DECLARATION

We hereby declare that the project entitled, **"CIRCUITRON: Circuit Recognition, Transformation, and Analysis"** in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering **Electronics, Communication and Information Engineering** is a bona fide report of the work carried out by us. These materials contained within the report have not been submitted to any University or Institution for the award of any degree, and we are the only author of this complete work and no sources other than the listed ones have been used in this work.

Anveshan Timsina (THA078BEI005)
Sandesh Dhital (THA078BEI034)
Saroj Nagarkoti (THA078BEI039)
Utsab Dahal (THA078BEI046)

Date: March 2026

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, a major project work entitled **"CIRCUITRON: Circuit Recognition, Transformation, and Analysis"** submitted by **Anveshan Timsina, Sandesh Dhital, Saroj Nagarkoti , Utsab Dahal** in partial fulfillment for the award of Bachelor of Engineering in Electronics, Communication and Information Engineering. The project was carried out under the special supervision and within the time prescribed by the syllabus.

We found that the students to be hardworking, skilled and ready to undertake any related work to their field of study and hence we recommend the award of partial fulfillment of Bachelor's Degree of Electronics, Communication, and Information Engineering.

Project Supervisor
Er. Kobid Karkee
Department of Electronics and Computer Engineering
Thapathali Campus, IOE

External Examiner
Er. Deepesh Prakash Guragain
Department of Electronics and Communications Engineering
Nepal Engineering College

Head of Department
Er. Umesh Kanta Ghimire
Department of Electronics and Computer Engineering
Thapathali Campus, IOE

March 2026

COPYRIGHT

ACKNOWLEDGEMENT

First, we would like to thank the Department of Electronics and Computer Engineering for providing us with an opportunity to learn and research on this project. We appreciate the department's assistance and recommendations in the selection and hopefully the successful completion of this project.

Special thanks to our supervisor, **Er. Kobid Karkee**, for his guidance, invaluable feedback, and constant support in regards to the project. His expertise and encouragement have been pivotal in shaping the direction, scope and implementation of this project.

Additionally, we would like to thank all of our teachers, friends, and other direct as well as indirect contributors for their indispensable support and encouragement during our study and execution of the project. We are also grateful to the authors of various papers that we have referenced for our project.

Anveshan Timsina (THA078BEI005)
Sandesh Dhital (THA078BEI034)
Saroj Nagarkoti (THA078BEI039)
Utsab Dahal (THA078BEI046)

ABSTRACT

CIRCUITRON is an integrated system designed to reduce the gap between typical hand-drawn electrical schematics and digital circuit simulation tools. The project pipeline transforms hand-drawn circuit diagrams into CircuitJS-compatible text files and enables the users to simulate and analyze circuits in an editable digital environment. Deep learning models such as YOLOv7 for component detection, two kinds of OCR models, TrOCR and also a custom OCR pipeline for text recognition, work together with BFS-based path finding wire detection algorithm to achieve the project's objectives of visualizing circuits as editable schematics, and allowing simulation. The system has achieved a mAP@0.5 of 95% for component detection, and a character accuracy of 84.5% for text recognition, and proper wire and node detection, for horizontal, vertical as well as diagonal wires, across varied drawing styles. A user-friendly interface allows manual corrections, schematic editing, and direct integration with CircuitJS for simulation. Additionally, the platform incorporates a DeepSeek v3.1-powered assistant to answer circuit-related queries, thus reinforcing learning through natural language interaction.

Keywords: circuit analysis, Multi-head BFS, simulation, TrOCR, YOLOv7.

TABLE OF CONTENTS

DECLARATION . . . . . **i**

CERTIFICATE OF APPROVAL . . . . . **ii**

COPYRIGHT . . . . . **iii**

ACKNOWLEDGEMENT . . . . . **iv**

ABSTRACT . . . . . **v**

LIST OF FIGURES . . . . . **viii**

LIST OF TABLES . . . . . **ix**

LIST OF ABBREVIATIONS . . . . . **x**

1 Introduction **1**

1.1 Background Information 1

1.2 Motivation 2

1.3 Problem Statement 2

1.4 Project Objectives 3

1.5 Project Scope and Applications 3

1.6 Report Organization 4

2 LITERATURE REVIEW **5**

3 REQUIREMENTS ANALYSIS **8**

3.1 Software Requirements 8

3.2 Feasibility Study 11

4 SYSTEM ARCHITECTURE AND METHODOLOGY **13**

4.1 Block Diagram 13

4.2 Line Detection 22

4.3 Use-Case Diagram 25

4.4 Performance Metrics Expectation 26

4.5 System Flowchart 27

4.6 Dataset Analysis 28

5 IMPLEMENTATION DETAILS **31**

5.1 Dataset Manipulation and Preprocessing 31

5.2 Line Detection via Path Finding 39

5.3 Web App Implementation and Interaction 43
5.4 Error Handling 44

6 RESULTS AND ANALYSIS 46
6.1 Comparative Analysis of Object Detection using YOLOv5 and YOLOv7 46
6.2 Class Taxonomy Consolidation 47
6.3 Holistic Object Detection Comparison 51
6.4 Retrained YOLOv7 Demonstration 54
6.5 Custom OCR Results and Analysis 55
6.6 Comparative Analysis of Optical Character Recognition (OCR) Models 56
6.7 OCR Demonstration 58
6.8 Junction-Based Wire Detection 61
6.9 Visualization of Line Detection via Path Finding 62
6.10 Combined Overlay of YOLOv7, TrOCR, and Line Detection Results . . 64
6.11 Frontend Visualization 65

7 FUTURE ENHANCEMENTS 67
7.1 Improvement in the OCR accuracy 67
7.2 Increment in the number of supported components 67
7.3 Authentication and Database Functionalities 67
7.4 Improvement in the frontened and general user experience 67

8 CONCLUSION 69

9 APPENDICES 70
Appendix A: Project Budget 70
Appendix B: Gantt Chart 71

REFERENCES 72

LIST OF FIGURES

Figure 4-1 Block Diagram of the CIRCUITRON System 13
Figure 4-2 Architecture of YOLOv7 15
Figure 4-3 Text Detection and Recognition Pipeline 17
Figure 4-4 Architecture of TrOCR 18
Figure 4-5 Use-Case Diagram for CIRCUITRON System 25
Figure 4-6 System Flowchart 27
Figure 5-1 Interaction Diagram for the Web Application 44
Figure 6-1 mAP@0.5 comparison over 100 training epochs. 46
Figure 6-2 Performance metrics comparison: (a) Precision-Recall and (b) F1-Confidence curves. 47
Figure 6-3 Comparison of training loss (four YOLO architectures) 48
Figure 6-4 Comparison of validation loss (four YOLO architectures) 48
Figure 6-5 Performance metrics vs epochs on the consolidated dataset 49
Figure 6-6 Final-epoch performance metrics for four You Only Look Once (YOLO) architectures on the 15-class dataset 50
Figure 6-7 Effect of class consolidation on YOLOv7 performance 51
Figure 6-8 Comparison of performance of all object detection models 52
Figure 6-9 Radar chart comparing the three best detection configurations across four performance metrics 52
Figure 6-10 Confusion Matrix for the retrained YOLOv7 53
Figure 6-11 Retrained YOLOv7 Result (i) 54
Figure 6-12 Retrained YOLOv7 Result (ii) 54
Figure 6-13 Training Loss vs Epochs (Custom OCR) 55
Figure 6-14 Custom OCR Performance Metrics and their Distributions 56
Figure 6-15 Performance metrics comparison, in percentage 57
Figure 6-16 Text Detection and Recognition using Custom OCR(i) 58
Figure 6-17 Text Detection and Recognition using TrOCR(i) 59
Figure 6-18 Text Detection and Recognition using Custom OCR(ii) 59
Figure 6-19 Text Detection and Recognition using TrOCR(ii) 60
Figure 6-20 Otsu's Global Thresholding 61
Figure 6-21 Bounding Box Overlay 62
Figure 6-22 Skeletonized Image with Detected Endpoints 62
Figure 6-23 Adjacency graph constructed using Multi-Head BFS algorithm . . . 63
Figure 6-24 Combined Results of YOLOv7, TrOCR, and Line Detection 64
Figure 6-25 Digitally Drawn Circuit (Color Inverted) 65
Figure 6-26 Fullscreen Capture of the Frontend 65

LIST OF TABLES

Table 4-1 Expected Performance Metric Values for Individual Modules 26
Table 4-2 Summary of Data in the CGHD Dataset 28
Table 4-3 Frequency of Each Class with its Code in the CGHD Dataset 30
Table 5-1 Consolidation of Original Classes into Unified YOLO Classes 35
Table 5-2 Time complexity breakdown for Multi-Head BFS algorithm 41
Table 5-3 Space complexity breakdown for Multi-Head BFS algorithm 41
Table 6-1 Comparison of Performance Metrics for YOLOv5 and YOLOv7 . . . 47
Table 6-2 Final epoch performance comparison of four YOLO architectures on the consolidated dataset 49
Table 6-3 Effect of class consolidation on YOLOv7 performance 50
Table 6-4 Comparison of performance of all object detection models 51
Table 6-5 Performance comparison of OCR models 57
Table 9-1 Project Budget Breakdown 70
Table 9-2 Project Timeline (Gantt Chart) 71

LIST OF ABBREVIATIONS

AC	Alternating Current
AI	Artificial Intelligence
API	Application Programming Interface
BJT	Bipolar Junction Transistor
CER	Character Error Rate
CGHD	Circuit Graph Handwritten Diagrams
CNN	Convolutional Neural Network
CRNN	Convolutional Recurrent Neural Network
CTC	Connectionist Temporal Classification
DC	Direct Current
EDA	Electronic Design Automation
FET	Field-Effect Transistor
FSM	Finite State Machine
GNN	Graph Neural Network
GUI	Graphical User Interface
HSV	Hue, Saturation, Value
IDE	Integrated Development Environment
IoU	Intersection over Union
JSON	JavaScript Object Notation
LLM	Large Language Model
LMDB	Lightning Memory-Mapped Database
LSTM	Long Short-Term Memory
LTspice	Linear Technology Simulation Program with Integrated Circuit Emphasis
mAP	Mean Average Precision
ML	Machine Learning
NMS	Non-Maximum Suppression
OCR	Optical Character Recognition
PCB	Printed Circuit Board
RCNN	Region-based Convolutional Neural Network
REST	Representational State Transfer
RF	Radio Frequency
RMS	Root Mean Square
SGD	Stochastic Gradient Descent
SPICE	Simulation Program with Integrated Circuit Emphasis
VQA	Visual Question Answering
YOLO	You Only Look Once

1 INTRODUCTION

"CIRCUITRON: Circuit Recognition, Transformation, and Analysis" is a system to convert hand-drawn electrical circuit diagrams into CircuitJS-compatible text files enabling simulation and graphical analysis along with digital visualization of the circuit in an editable schematic. The system also integrates a prompt-based Artificial Intelligence (AI) assistant (DeepSeek v3.1) to help answer queries related to the relevant circuit.

1.1 Background Information

The first step of learning about electrical and electronic circuits involves looking at and hand-drawing said circuits with the necessary components. These hand-drawn circuit diagrams continue to play an important role in electronics education and early-stage hardware design due to the convenience they offer. However, such hand-drawn diagrams prove to be limited in their utility in the current education landscape that prioritizes intuitive, visual understanding as they require significant mental gymnastics to facilitate any form of analysis of the circuits, more so when incorporating variation in parameters.

A slew of rather easy-to-use circuit simulation and analysis tools such as Linear Technology Simulation Program with Integrated Circuit Emphasis (LTspice) and KiCad exist, of course; and they continue to foster visual learning and analysis of circuits of a range of complexities. But these traditional Electronic Design Automation (EDA) tools lack the capability to accept hand-sketched circuit diagrams as input. Instead, users are required to redraw the circuits in software using Graphical User Interface (GUI) tools or by manually entering Simulation Program with Integrated Circuit Emphasis (SPICE) netlists; steps that are both tedious and error-prone [1, 2]. Be it due to the (admittedly not very steep yet inconvenient) learning curve of these tools or just the additional effort of having to redraw a circuit digitally (the dullness of which only goes up with circuit complexity), the instructors as well as the students often tend to neglect arguably essential simulation-based intuitive teaching and learning. This again encourages the same long-established rote learning practice instead of ushering in a more modern way of understanding circuits and electronics at large. It has been long proven that students tend to perform better when given the appropriate simulative manipulation and concept clarification tools, especially in the context of electrical and electronics circuits [3]. It then follows that the reluctance to use such tools is actively hindering the students' potential. Thus, the use of visual manipulative simulation tools is the need of the hour. In fact, it has been the need of the hour for quite a while. But, due to a lack of any uber-convenient method, it is yet to be popularly embraced, primarily in technologically nascent countries such as Nepal.

1.2 Motivation

In recent years, the mantra of the modern world has seemingly become “adapt or die”. The society regularly stomps on the tech-illiterate to make way for the ones willing to champion the use of new and shiny tools in all aspects of their lives. In an increasingly globalized economy, this is especially true for the up-and-coming students who need to compete with other students all over the world relying solely on their understanding of subject matter in their particular fields. In such case, it becomes essential that they are given the tools to enhance their learning and understanding even by minor increments. In the context of electronics, circuits are the fundamental building blocks that permeate each and every aspect of our “teched-out” world and it is important for learners in the field to have an intuitively in-depth understanding of this fundamental ingredient. Thus, there exists a motivation for a hassle-free method to analyze and understand circuits to deepen the necessary conception and comprehension.

The motivation stems not only from a deep-seated commitment to give the instructors and students tools to easily visualize and analyze circuits in the belief that it will genuinely improve the quality of professionals and engineers born out of universities, but also from the personal experience of the members involved in this project. As the electronics engineers of the near future who have had and will continue to have to deal with circuits in one form or another, the members involved in this project are uniquely positioned to understand the plight of a beginner in the study of electronics. And a tool which can convert a hand-drawn circuit into a digital schematic and also allow for analysis would go a long way in alleviating that plight.

1.3 Problem Statement

The EDA tools currently available do not support the direct transformation of hand-drawn circuit images into simulation-ready schematics. There exist a few nascent prototypes for the same but there seems to be no widely adopted, robust, and user-friendly pipeline that combines component recognition, reliable wire connectivity, schematic generation and simulation, whether by itself or by integration with SPICE simulators.

“CIRCUITRON: Circuit Recognition, Transformation, and Analysis” project aims to create an integrated system capable of first, recognizing and digitizing hand-drawn circuits into editable schematics and then allowing the user to analyze those circuits through graphs using CircuitJS-compatible text files with an added AI assistant to handle related queries.

1.4 Project Objectives

- To automatically convert hand-drawn circuit diagrams into CircuitJS-compatible text files for simulation.
- To visualize the circuit in an editable schematic in addition to an AI assistant for handling simple, relevant queries.

1.5 Project Scope and Applications

1.5.1 Project Scope

This project focuses on the development of a software pipeline that accepts scanned or photographed images of hand-drawn electrical circuits and transforms them into CircuitJS-compatible netlists for simulation as well as editable schematics through an intermediate integrated data structure to facilitate the transformation. The scope includes detecting common circuit components (14, to be exact, including supplementary components like gnd, crossover, etc.) and their values in mentioned units. It infers wire connectivity with junction recognition and generates a circuit diagram with simulation capabilities. Additionally, the system incorporates an Large Language Model (LLM)-based (Deepseek v3.1) query-solving assistant. The system excludes any form of advanced circuits such as those associated with Radio Frequency (RF) or mixed-signal, and Printed Circuit Board (PCB) designs.

1.5.2 Project Applications

- The system is foremost, an educational tool designed to assist students and instructors alike in conveniently digitizing circuit diagrams and understanding circuit behavior through limited simulation and interactive Q&A.
- The project allows for quick feedback and validation of sketched designs against simulation results.
- The project can facilitate swift analysis of circuits for students and potentially, field engineers as well.

1.5.3 Project Limitations

The system is constrained by several factors in spite of its innovative design. The detection and recognition accuracy depends heavily on how clear the input images are. Low-resolution images or messiness in circuit sketches may lead to recognition errors. Also, the system supports only a fixed set of common components (based on the currently used datset) which means it cannot handle rare components and hence has limited functionality. Wire connectivity mapping is achieved through a junction-based

approach, which assumes standard drawing practices. However, there is no one way to draw a circuit diagram; non-standard and overlapping wires, or complicated circuits may confuse the system. Additionally, the system lacks a simulation engine of its own and instead uses an open-source tool like CircuitJS. This limits the simulation capabilities to that of CircuitJS. More rigorous training on appropriate data would help remedy some of these issues, however finding the ideal dataset with enough handwritten examples for all kinds of components and circuits has proven challenging.

1.6 Report Organization

The report begins with an Introduction which includes background information, motivation, problem statement, project objectives, applications, and limitations. Literature Review presents a summary of existing relevant research and their corresponding limitations that are in turn addressed by this project. Following this, Requirement Analysis simply describes the project requirements i.e, software needs, on account of it being a software-only project, along with feasibility study (economic, time, operational, technical). System Architecture and Methodology explains the system design through diagrams like block diagram, flowchart, use-case diagram, and architecture diagrams for specific modules used in the project. These diagrams are supplemented with serviceable explanations for each element in the project pipeline. It also includes dataset analysis and performance metrics expectation tables. Implementation Details elucidates practical aspects of the system: preprocessing techniques and model training details. It also includes relevant algorithms for wire detection, wire connectivity, and error handling techniques. Results and Analysis part details project outcomes giving the results of all individual elements (object detection, text recognition, line detection), in addition to performance analysis of different models used (with the use of graphs). Future Enhancements put forward potential improvements or extensions to the project's features or performance metrics. Conclusion presents a summary of the entire project focusing on if and how the objectives have been achieved. Appendices contain budget breakdown and Gantt chart for the project.

2 LITERATURE REVIEW

Hand-drawn circuit recognition and digitization, while not a vastly researched area, has a few notable examples of its implementation. Most of the study and implementation related to this has been done on classifying the different circuit components (like battery, resistors, inductors, etc.) and there's not much on arguably the more difficult aspect of circuit recognition: node and wire connectivity. For instance, in a 2021 study, 20 different components were classified from self-made hand-drawn circuits with the recognition accuracy of 97.33% [4].

In one of the oldest studies on the automatic recognition of hand-drawn circuits, an approach of identifying the components first and then the node segments (taking into consideration their spatial relationship) was implemented, achieving an accuracy of 86% for components and 92.5% for the nodes. The node segments were then traced assuming wire between them, considering a variety of hard-coded conditions [5]. In a recent 2024 research, a couple of UK engineers were successful in digitizing images of electrical circuit schematics using a novel pattern-recognition methodology and custom-trained OCR model achieving respective success rates of 86.4% and 99.6% [6]. This, however, was trained on printed/computer-made circuit diagrams, not hand-drawn ones and also isn't able to understand the values of the circuit components.

To overcome the limitations of Convolutional Neural Networks (CNNs) in understanding circuit topology, graph-based methods have gained traction. Circuit2Graph, a pipeline that transforms circuit diagrams into graph structures where nodes represent components and edges denote electrical connections was introduced in 2023. Using Graph Neural Networks (GNNs), their system achieved 97.9% accuracy on circuit classification tasks, outperforming traditional methods [7]. This approach demonstrated that GNNs can effectively model the global structure of a circuit, even when visual features alone are insufficient. Similarly, Bayer et al. [8] used Mask Region-based Convolutional Neural Network (RCNN) for instance segmentation of components in hand-drawn diagrams, followed by graph extraction techniques to infer connections. Their model achieved around 94% mask accuracy, but they noted that symbol recognition was "reasonable, yet incomplete," especially in more complex drawings or where handwriting was less consistent. A paper published in the International Research Journal of Engineering and Technology proposed a two-step method for electronic circuit diagram detection and recognition; first detecting and classifying the components present in the circuit and then labeling the classified components to form a string which is then fed to an Finite State Machine (FSM) system for circuit recognition [9].

With the use of YOLOv5 for detection of circuit components and Probabilistic Hough transform-based solution for node recognition and taking in hand-drawn circuits, a couple

of Indian engineers were successful in achieving not only a Mean Average Precision (mAP) of 98.2% in component detection but also a near-real time generation of circuit schematic with an accuracy of 80% [1]. However, their circuits were not labelled which is to say the values for the battery (in volts), resistance (in Ohms), etc. were not written in the circuit. Another very recent study suggested a three-step process to detect and identify the circuit and its components with the steps including detection of components, detection of lines and their junctions and then the detection of text placed near the components and then combining the data from these three streams into a JavaScript Object Notation (JSON) format to eventually derive a netlist from [10]. This is a similar workflow to the one this project intends to use; however, the scope of our project is much larger owing to the fact that an editable schematic is to be delivered on top of taking in hand-written circuit diagrams as opposed to generating a netlist from a digital schematic.

Netlist generation is also a very critical aspect of this domain. While traditional EDA tools such as KiCad and LTspice support netlist export from manually created schematics, they do not allow direct image-to-netlist conversion [11]. This limitation was addressed with Img2Sim; an artificial neural network-based system that detects components, infers connectivity, and outputs a complete SPICE netlist. Their model reported over 90% accuracy in topology extraction and represents one of the first functional pipelines to translate a hand-drawn circuit into a ready-to-simulate format [2]. However, Img2Sim and similar systems typically require clean input images and are limited to a predefined set of components, reducing their robustness in real-world use cases. While the feasibility of such pipelines is now evident, a generalized, accurate, and open-source solution that goes the extra mile to also visualize the circuits digitally remains unavailable.

Parallel to recognition and netlist generation, visual question answering (Visual Question Answering (VQA)) for circuits is emerging as a useful educational application. CircuitVQA is a dataset containing more than 115,000 question-image pairs based on 5,725 unique circuits. It covers approximately 70 symbol types and supports the training of transformer-based VQA models capable of answering questions like “What is the function of R1?” or “Is this a voltage divider?” Early experiments demonstrated that models fine-tuned on CircuitVQA outperformed generic VQA systems, indicating the importance of domain-specific training. Meanwhile, large language models (LLMs) such as GPT-4 have also been explored for circuit explanation tasks. For instance, the SPICEPilot framework attempts to generate and interpret SPICE code using GPT-based models [12]. However, these general-purpose models often fail to respect domain conventions, such as valid W/L ratios or simulation types, resulting in inaccurate outputs.

Finally, simulation integration plays a vital role in validating and interacting with generated circuits. Ngspice, an open-source simulation engine, is widely used in

research and is natively supported in KiCad. It enables Direct Current (DC), Alternating Current (AC), and transient analysis, making it suitable for most educational and prototyping needs. In contrast, commercial tools like Simulink and PSpice provide broader capabilities but are cost-prohibitive. Some research efforts, including Img2Sim, have leveraged open standards to generate SPICE files compatible with tools like Ngspice or PSpice. Other projects utilize Python-based libraries such as PySpice to enable automated simulation directly within a code-driven environment as well as the web-based CircuitJS [13], which is exactly how this project intends to fulfil its simulation needs.

While significant progress has been made in component recognition and value comprehension, connectivity inference, netlist generation, and interactive Q&A, existing systems remain fragmented or limited in scope. An end-to-end system that combines all these elements into a user-friendly platform remains a critical need for both educational and practical applications.

3 REQUIREMENTS ANALYSIS

3.1 Software Requirements

3.1.1 Python Libraries

- **PyTorch and TensorFlow:** PyTorch is an optimized tensor library useful for deep learning using CPUs and/or GPUs. It provides a industry-standard platform for training and deploying DL models with added tools and libraries. TensorFlow is a similar platform to PyTorch with similar capabilities; in fact, these two exist and are used simultaneously, preferring one over the other for specific tasks. TensorFlow specifically simplifies the ML/DL workflows to run in any environment. For this project which juggles different ML/DL models which need to be trained and deployed, these libraries are extensively used.
- **Numpy:** Numpy is marketed as the fundamental Python package/library for scientific computing. It is an interoperable open-source library which is very optimized for multi-dimensional array computations. ML models break-down all elements as large array of numbers and Numpy comes very handy in facilitating all sorts of numerical computations and other mathematical operations on these arrays. It is used in all aspects of image processing in this project.
- **OpenCV:** OpenCV or Open Source Computer Vision Library, as the name suggests, is an open-source Python library that includes various computer-vision algorithms. It defines basic data structures for images, provides modules for image processing, includes functions for reading and writing files in various image formats, and even provides features for working with video and 3D animation. It also offers a hyper-optimized OCR functionality, however this is not applicable for hand-written text. OpenCV working together with Numpy in this project has been used for image processing tasks like thresholding, augmentation, etc. Any time an image is visualized after feeding it to a DL model, OpenCV is most likely used.
- **Pandas:** Pandas is an open source data analysis and manipulation tool for Python. It provides labeled data structures (useful for user-defined data too) and statistics-based functions. It is primarily used while working with relational or labeled data (which this project uses a plethora of).
- **Matplotlib and Seaborn** Matplotlib is a Python library for creating static or animated visualizations (graphs, charts). It even allows for interactive graphical representations. All the graphs that showcase performance of trained models have been generated using Matplotlib. Also, in initial testing phase, it was used to generate circuit-related graphs (simulation of say, voltage vs time), first simply static and then later, interactive. Seaborn is a high-level data visualization library based on Matplotlib. It is used to

make the visualizations more attractive and informative. It was used in a limited capacity during testing as Matplotlib's features mostly were adequate for this project.

- **Scikit-learn:** Scikit-learn comes in during the post-processing stage. It provides tools for predictive data analysis. It provides easy implementation of k-means algorithm (especially useful for this project), among many others. Although used in a rather limited capacity in the project, it proved useful for clustering of junction points. It has also been used for the evaluation of the models during and after training.
- **Hugging Face:** Hugging Face provides a large hub of pretrained transformer models for Natural Language Processing (NLP), vision, and more. It is used in the project to integrate LLMs for answering circuit-related queries and working with TrOCR.

3.1.2 YOLOv7

The YOLOv7 model is used to detect and classify circuit components in hand-drawn circuit images. It outputs bounding boxes, class labels, and confidence scores for each detected component. After comparison with other version of YOLO (v5, v8, v11), YOLOv7 is selected due to its superior detection accuracy, efficient training capabilities, and support for multiple object scales through its enhanced architecture, while also being of an appropriate size in regards to speed and training time.

3.1.3 OCR models

A CRNN-based OCR model trained from scratch, following the DTRB design philosophy for recognizing text is vital for the project as it is the core technology which reads and understands the component values and units of circuit elements. This has been chosen for the project because, foremost, other ready-made OCR models proved to be either too resource-hungry and/or failed to perform well even after some fine-tuning. Additionally, a minimally fine-tuned TrOCR model has also been used. The uses essentially gets to choose between a faster but slightly inferior custom OCR model, or a Microsoft-built robust yet slower TrOCR.

3.1.4 KiCad Symbol Library

KiCad is a free, open-source EDA suite which offers its libraries to be used freely to build any educational project. The project uses the KiCad symbols library, specifically, in order to simply extract the component symbols standardized by KiCad rather than building the icons/representations for components from scratch for use in the frontend schematic.

3.1.5 Coding and Training Platforms

- **Google Colab:** Google Colab is a cloud-based platform that offers a Jupyter notebook environment for developing deep learning systems. For this project, Google Colab provided with Graphics Processing Units (GPUs) for free, which significantly accelerated the training process, enabling experimentation with larger models and datasets. Google Colab has in-built Pytorch and TensorFlow libraries, which made it easy to use them online with minimal setup (barring the occasional, headache-inducing version incompatibility issues). It also allows for Google Drive to be its storage for zipped datasets that need to be used in the notebook.
- **Kaggle:** Kaggle is a multi-faceted online platform popular for data science competitions, datasets, code sharing and model training. It was primarily used to find relevant datasets (Circuit Graph Handwritten Diagrams (CGHD), VQA, Handwritten Circuit Diagram (HCD)), code sharing and model training. The dataset of choice for this project was also found in Kaggle. Additionally, it provided a fully hosted environment with popular Python libraries such as Pandas, Numpy, Tensorflow, PyTorch pre-installed to make implementing and training models convenient, especially with its integration with Google Colab. Its free cloud computing service providing 30 hours of GPU use per week proved indispensable in accelerating the training and testing of different models.
- **Lightning AI:** Lightning AI is a cloud workspace, similar to Google Colab and Kaggle, which specializes in helping build agents and train models without any tedious setup. Most importantly, through the use of a student account, it allows for the use of extremely powerful GPUs for free (for a reasonable amount of time). Among all the cloud computing services used for the project, Lightning AI proved the most invaluable for implementing and training the models as the GPUs it allotted were super-fast and saved a ton of time during the tedious and time-consuming process.
- **Visual Studio Code:** VS Code is the local Integrated Development Environment (IDE) everyone on the team settled on, mainly for its auto-completion, syntax highlighting, and copilot integration. Each member extends it further with their own set of extensions tailored to their workflow. All locally executed code runs through VS Code, and its usefulness increased ten-fold once the web development phase began.

3.1.6 Web Development

- **Reactjs:** Reactjs serves as an integral technology underlying the dynamic front-end interfaces to be developed for the project. The frontend for this project includes dynamic elements like dragging, erasing, resizing, etc. of the circuits and circuit elements, and Reactjs fulfills these requirements with convenience and speed through the use of its features like virtual Document Object Model (DOM) abstraction,

distinction of code into reusable logic and presentation buckets, etc. as well as compatible packages like react-draggable.

- **Next.js:** Next.js is a full-stack framework to build web apps. It is built on top of React by adding server-side rendering and static rendering. While Next.js' main selling point is server-side rendering, it also simplifies frontend workflows that are done using React. The client-side interface for this project is built primarily using Next.js.
- **FastAPI:** FastAPI is a backend web framework for building APIs based on standard Python type hints. It offers performance on par with NodeJS. It used Starlette (for the web parts), and Pydantic (for the data parts). The backend of this project, especially relating to handling the data, is done using FastAPI and Pydantic. Pydantic is a data validation library for Python that is also powered by type hints, hence making it convenient to be used with FastAPI. Pydantic also checks the data moving between backend and frontend based on predefined structures.
- **Uvicorn:** Uvicorn is an ASGI web server implementation for Python. This means it handles HTTP requests asynchronously. The FastAPI standard installation itself includes Uvicorn which is needed to run the application.
- **Github:** Github is a proprietary developer platflorm that uses Git to provide distributed version control. This project uses Github for storing and sharing the project code and resources among project members.

3.2 Feasibility Study

3.2.1 Technical Feasibility

Technical Feasibility is assessing if a project idea can be built with the team's current (or selected) technology and resources. It also assesses whether or not the project is within the skill-level of the project team. Foremost, the technology used for the project has been selected after consideration of their features and support. Python libraries such as TensorFlow, Numpy, OpenCV, Matplotlib, etc., React and Netxt.js for frontend, FastAPI with Pydantic and Uvicorn for backend, are all mature, ever-developing technologies and can handle the requirements for this project. They have been selected with the expectation that these technologies will be supported for a considerable time in the future. They also have excellent docs and community support behind them, useful for handling the inevitable issues that arise during development and testing. The project members are decent developers with considerable experience in research and Python-based developement. Overall, the project is technically feasible.

3.2.2 Economic Feasibility

The total expenses for this project have come out to be NPR 41,160. The biggest part of this is the subscription for Google Colab Pro and Lightning AI credits, both useful in training ML/DL models (NPR 24,000). Lightning AI provides token-based scaling and also a generous free tier which is why it was used more than Google Colab Pro. Plus, a lot of research has gone into this project, and the scientific papers required for this were made available through an IEEE Xplore Membership amounting to NPR 2,160. Miscellaneous costs such as report printing and presentation costs as well as API costs, inevitable to any software-based research project amount to 15,000. The total cost is considered justified for a project of this scale and caliber, and hence is economially feasible. Gantt Chart in the Appendix expands on this.

3.2.3 Time Feasibility

The project has taken almost a year to complete by a team of four members. This was expected as it has a lot of moving parts and modules. Before any development, around a month was dedicated to preliminary research and literature review. The scope of this project changed in subtle ways during this, depending on the shortcomings of the existing research and products. This research, however, did not stop once development started, rather, it took a background role throughout the project, helping shape the project.

Phased development approach was followed. Phase A covered the circuit recognition (YOLO), text extraction (custom-OCR and TrOCR), and netlist generation. Phase A was developed and tested by September of this year. A buffer period was planned between Phases A and B for adjustments.

Phase B broadened the feature list. Integration of all elements of Phase A was done into a barebones web application. CircuitJS was roped in for handling the editing of schema and simulation needs of the project. DeepSeek v3.1 was utilized for the LLM-based query feature. Gradual, iterative developments in the UI/UX and subsequent testing continued in the background. This phased approach kept the design intact while layering on the more advanced features over time.

As of yet, all of the promised features have been delivered with only minor kinks to be sorted out. The project is considered feasible from a time perspective.

4 SYSTEM ARCHITECTURE AND METHODOLOGY

4.1 Block Diagram

The process of converting a handwritten circuit diagram into a structured netlist involves a series of coordinated steps, each step feeding into the next, designed to detect, extract, and represent pictorial information in a machine-readable format and then using this data for simulation and schematic generation.

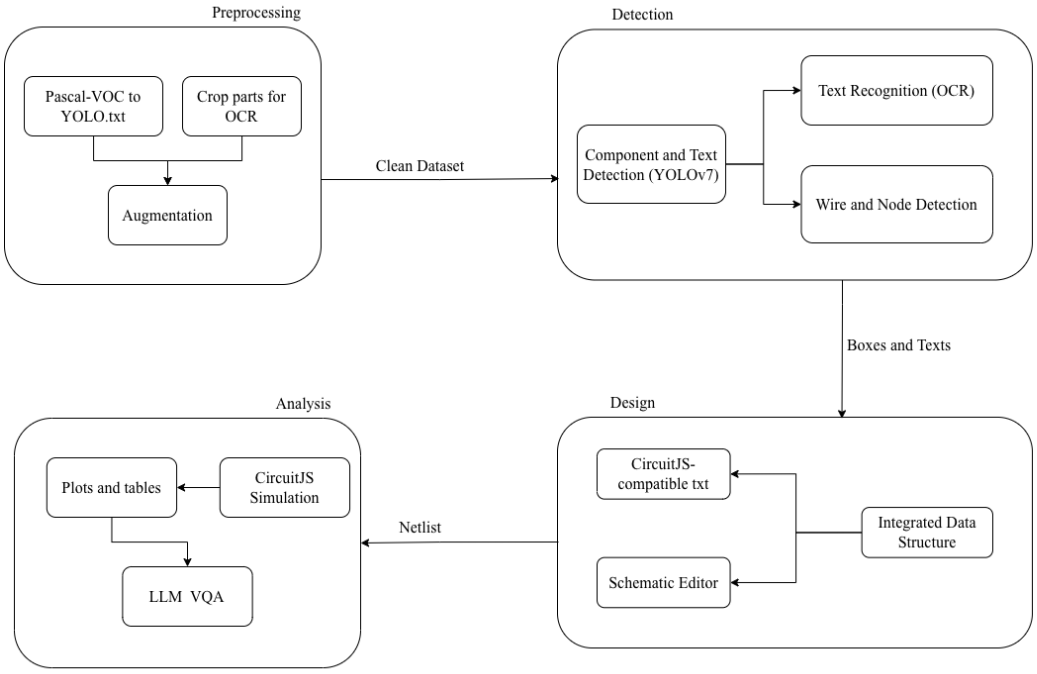


Figure 4-1 Block Diagram of the CIRCUITRON System

The high-level architecture of the CIRCUITRON system is visualized in the block diagram in Figure 4-1. Each step involved in the hand-drawn image to simulation-ready schematic pipeline is elaborated as follows.

4.1.1 Preprocessing

At first, the circuit information (components, component values, units, wire intersection, etc.) needs to be extracted out from the hand-drawn image. Appropriately fine-tuned ML and DL models are to be used for each of these steps which will be elaborated on in later subsections. However, for any sort of detection to happen, the image needs to first be pre-processed. Preprocessing for this not only includes image manipulation tasks but also conversion of the dataset annotations into appropriate formats to maintain compatibility with the component detection and text recognition models. The dataset of

choice is in the Pascal VOC format which is incompatible with the models used in the pipeline and hence need to be converted.

Furthermore, image preprocessing, which includes steps like thresholding, contrast and brightness enhancement (for improved clarity) is a vital step in readying the dataset for training any and all models. However, geometric augmentations, especially rotation, is not appropriate for this project.

Among the various available preprocessing techniques, the ones have been selected with careful consideration of the type of images available in the dataset as well as the models in use. Besides image manipulation, preprocessing also includes splitting the dataset into training, validation and testing subsets as appropriate.

4.1.2 Circuit Component Detection

A preliminary yet critical step in the project pipeline is the accurate detection of circuit components from hand-drawn schematic images. This is addressed using a real-time object detection algorithm which is capable of identifying and localizing multiple electronic components (such as resistors, diodes, etc.) directly within the image. YOLO algorithm, the YOLOv7 model, specifically, has been selected for the component detection needs of this project. It is to be noted that text, as a generic collection of letters, numbers, and special characters is also considered a component in this context.

YOLO (You Only Look Once) is a real-time object detection algorithm known for speed and efficiency. YOLO frames object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities. This model works as follows: first, dividing the image into an $S \times S$ grid. Each grid cell is responsible for detecting objects whose centre falls within it. For each cell, a number of bounding boxes and their corresponding confidence scores (which represents the accuracy of the box and whether the box contains an object) are predicted. Each predicted bounding box includes the coordinates of the box centre relative to grid cell, represented by (x, y), the width and height relative to the whole image, represented by (w, h) and the confidence score C given as:

$$C = P(\text{object}) \cdot \text{IoU} \tag{4-1}$$

where $P(\text{object})$ denotes the probability of an object being inside the box, and Intersection over Union (IoU) is the Intersection over Union between the predicted box and the ground truth box.

Additionally, for each box, class probabilities are predicted as:

$$\text{Class Score} = P(\text{class } i \mid \text{object}) \cdot C \tag{4-2}$$

This allows the model to classify which object exists in each box and once the predictions are made, Non-Maximum Suppression (NMS) is applied to discard less-relevant overlapping boxes.

Based on similar fundamental working principle, several iterations (versions) of YOLO have been introduced and popularized among which YOLOv7 is the YOLO model of choice for this project.

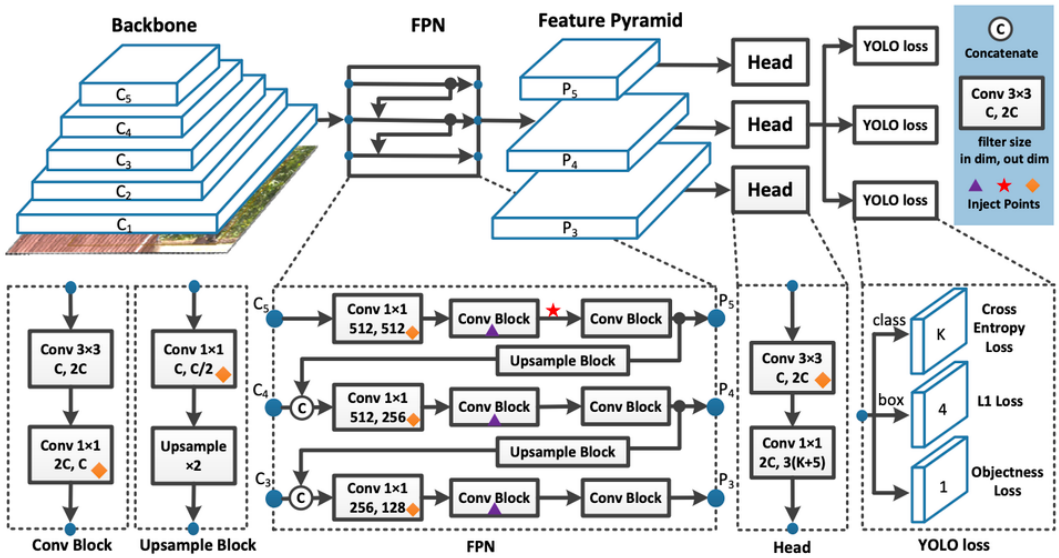


Figure 4-2 Architecture of YOLOv7

Figure 4-2 [14] shows the three primary modules of YOLOv7 architecture: Backbone, Neck, and Head, all with their own functions. The CSP-based SPPF backbone extracts essential features from input images. The neck is a PAN which combines features extracted by the backbone and passes them to the head for detection. The head predicts bounding box coordinates, confidence scores, and class probabilities for all the detected objects in a given image.

Prediction of the bounding box coordinates is done through anchor-based regression such that the final box $B = (x, y, w, h)$ is worked out relative to the anchor box as follows:

$$x = \sigma(t_x) + c_x \tag{4-3}$$

$$y = \sigma(t_y) + c_y \tag{4-4}$$

$$w = p_w \cdot e^{t_w} \tag{4-5}$$

$$h = p_h \cdot e^{t_h} \tag{4-6}$$

where (t_x, t_y, t_w, t_h) are the predicted offsets, (c_x, c_y) is the top-left corner of the grid cell, and (p_w, p_h) are the dimensions of the anchor box.

YOLOv7 runs auto-augmentation features (mosaic augmentation, HSV shifts, crops, rescaling, etc.) during training along with other features to increase dataset variability. Built-in early stopping is also there; if the validation loss (a mix of box loss, objectness loss, and classification loss) plateaus for a set number of epochs, training cuts off on its own to avoid overfitting.

4.1.3 Recognition of Component Values

In order to generate a complete and simulation-ready netlist, it is essential to not only detect the components themselves but also to extract their associated parameter values, which are generally annotated near the components in the circuit diagram. Thus, it becomes essential to integrate Optical Character Recognition (OCR) with the object detection pipeline to extract and map these values correctly to the corresponding components.

This involves identifying regions within the circuit image that likely contain text (done by YOLOv7), and then recognizing said texts in order to parse the component values along with their units.

The hand-written text recognition is done using a Convolutional Recurrent Neural Network (CRNN)-based OCR system, following the design philosophy of Deep Text Recognition Benchmark (DTRB) framework, however minor changes deemed suitable for this project have been made to get the text detection and recognition pipeline.

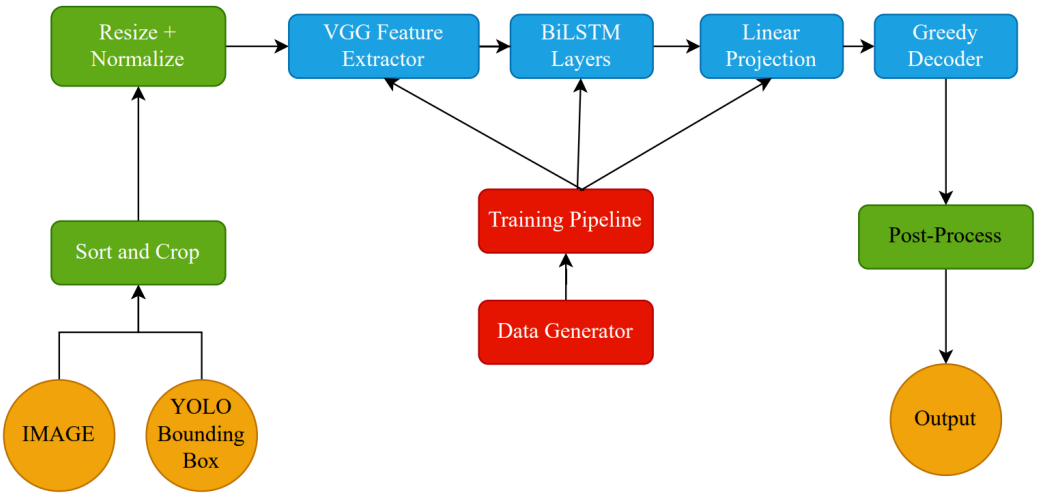


Figure 4-3 Text Detection and Recognition Pipeline

YOLOv7 outputs (bounding box information) help segregate textual information (class '0' in the used dataset) which are then cropped, resized (to image height as 64) and normalized for compatibility with text recognition module. This is then passed through to a CRNN architecture composed of a Convolutional Encoder, Sequence Modeling Engine with Linear Projection, and Connectionist Temporal Classification (CTC) Decoder as shown in Figure 4-3.

Convolutional Encoder (CNN): A CNN stack (e.g., VGG) transforms input image I from multi-dimensional $\mathbb{R}^{H\times W\times C}$ to feature maps:

$$F = f_{\text{CNN}}(I_i), \quad F \in \mathbb{R}^{H'\times W'\times C} \tag{4-7}$$

Sequence Modeling (Bidirectional LSTM): Each F becomes input to a bidirectional Long Short-Term Memory (LSTM), yielding hidden states h_t. These hidden states capture context across the text line, allowing modeling of character sequences:

$$\overrightarrow{h_t} = \text{LSTM}_f(F_t, \overrightarrow{h_{t-1}}) \tag{4-8}$$

CTC Decoder: A softmax layer projects h_t over the character vocabulary as:

$$p_{t,k} = \frac{e^{W_k h_t + b_k}}{\sum_{k'} e^{W_{k'} h_t + b_{k'}}} \tag{4-9}$$

The probability of sequence z is computed over all alignments:

$$P(z \mid y) = \sum_{\pi : B(\pi) = z} \prod_{t=1}^{T} p_{t,\pi_t} \tag{4-10}$$

The CTC loss is given by the negative logarithm of $P(z \mid y)$. It enables end-to-end sequence decoding without explicit character bounding boxes.

During inference, greedy decoding strategy is used which selects the most-probable character at each time step while removing blank symbols and collapsing consecutive duplicate characters. The recognized text then needs to be mapped to the appropriate circuit component whose value it represents.

This project includes two separate OCR models for character recognition, the second one being TrOCR (TrOCR-small to be exact) developed by Microsoft, specifically for handwritten text.

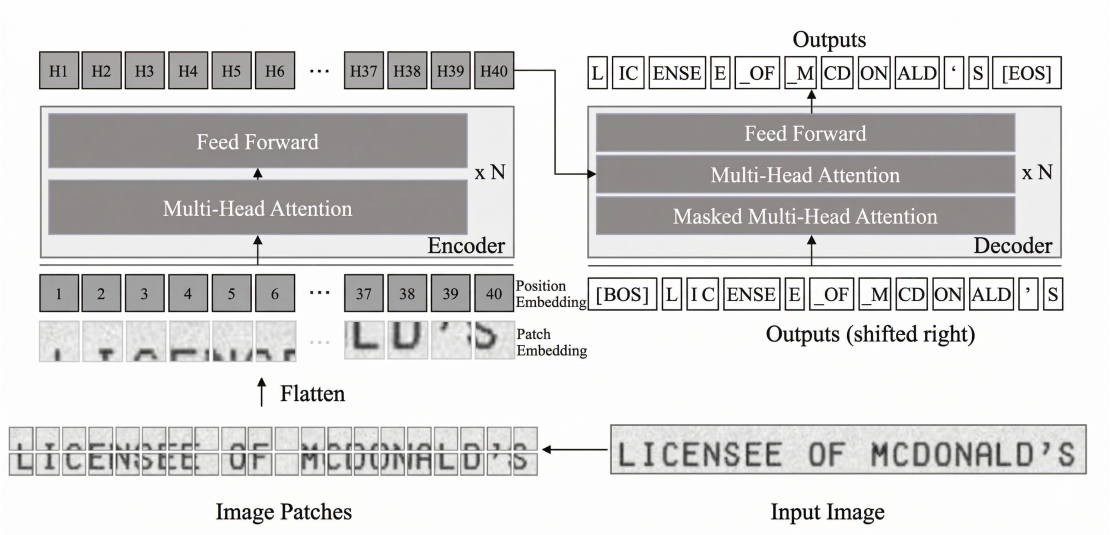


Figure 4-4 Architecture of TrOCR

Figure 4-4 [15] shows the encoder-decoder model (with pre-trained image transformer as encoder, and pre-trained text transformer as decoder) completing the basic architecture of TrOCR. The image transformer obtains the representaton of image patches, and likewise, with the help of visual features and previous predictions, the text transformer generates the wordpiece sequence.

The process begins with the raw input image, denoted as $x_{img} \in \mathbb{R}^{3 \times H_0 \times W_0}$. The transformer architecture is designed to process sequences rather than raw pixel grids, therefore the image is first resized to a standardized dimension (H, W).

To format this data for the model, the image is decomposed into a series of N fixed-size square patches. For P as the patch side length, the total number of patches is given as:

$$N = \frac{HW}{P^2}$$

Every patch gets flattened into a vector and then linearly projected into a D-dimensional space; D here is the hidden dimension that stays constant across all transformer layers.

A `[CLS]` token is prepended to the sequence so that global information can be pooled into a single representation. If pre-trained DeiT models are used for initialization, a distillation token gets tacked on as well to help the model learn from a teacher. Transformers on their own have no built-in sense of where patches sit spatially (unlike CNNs), so learnable 1D position embeddings are summed into the patch embeddings to retain absolute positional cues. Viewing the image as a sequence of standalone patches lets the model spread its attention across fine-grained details or the broader layout as needed.

The decoder follows the standard transformer decoder blueprint for producing the final text sequence. It mirrors the encoder's stacked-layer design, though an encoder-decoder attention block is wedged in between the self-attention and feed-forward portions. Queries in this block come from the decoder's own internal state, and keys along with values are drawn from the encoder output. That arrangement lets the decoder zero in on whichever visual features matter most at each generation step.

A self-attention mask keeps the model from peeking at future tokens while training. The output at any position i therefore depends solely on the tokens before it. Hidden states from the decoder pass through a linear layer sized to the vocabulary V, and the resulting probability distribution over the vocabulary comes from the softmax function:

$$\sigma(h_{ij}) = \frac{e^{h_{ij}}}{\sum_{k=1}^{V} e^{h_{ik}}}$$

At inference time, beam search is used to explore the probability space and land on the most probable character sequence.

4.1.4 Wire Detection

Junction points picked up by the object detection model act as structural anchors for figuring out how the circuit is wired. The detected junction coordinates are first converted from the normalized values into pixel coordinates. At this stage, which junctions are actually connected is still unknown, so each pair of junctions needs to be examined to see which ones could form wire connections. For each junction pair, the displacement and

the angle between them is computed and since most circuit diagrams use only horizontal and vertical wires, diagonal pairs are discarded. The remaining pairs of junctions are then forced into perfectly horizontal or vertical segments. As the detection is never perfectly accurate, the wire endpoints are snapped to a uniform grid in order to align slightly-off wires. Segments that lie on the same line and overlap with each other are also merged. After these corrections, we obtain a clean set of wire segments that accurately reflects the wiring drawn in the diagram in strictly horizontal and vertical planes. This is a limitation that is fixed by the BFS-based algorithm, discussed in later sections.

4.1.5 Terminal Point Detection

After detecting components (through YOLO) and wires, the exact points where they connect are still unknown. The junction points detected earlier give a good starting point for locations of wire connections, since they include wire endpoints and intersections.

To find the actual terminals, we check where horizontal or vertical wire segments enter (or touch) the component bounding boxes. Whenever a wire intersects with a bounding box, calculated using self-defined line-rectangle intersection rules, we treat that crossing as a terminal point. These terminal points reveal which wires are connected to which components. This information is then used to build the circuit graph and to display the connections correctly in the frontend.All components of the project (CircuitJS-compatible text file generation, schematic editing, etc.) build on this representation downstream.

4.1.6 Integration Algorithm

From the individual outputs from each of the aforementioned modules, the task is now to generate a structured JSON representation of the circuit which has is to be used to generate a corresponding CircuitJS-compatible text file and also contribute in the schematic generation further down the line.

The integration between all these different outputs, in a simplistic form, would consist of the following steps:

- Get a list of components and their corresponding bounding box coordinates (from YOLOv7).
- Get a list of text data and their corresponding bounding box coordinates (from OCR).
- Map the component to the corresponding component value (text) based on the minimum Euclidean distance between a particular component and texts on the circuit.
- Once the terminals are determined based on the bounding boxes, match those terminal points to the nearest nodes in the circuit.

- Store the accumulated information in a JSON file.

4.1.7 CircuitJS-compatible Text File Generation

A circuitJS-compatible text filedescribes the connections between components in an electronic circuit. It acts as a blueprint, specifying which components are connected and how, without detailing the physical layout or specific component placement.

It is a text-based representation of an electrical circuit used for simulation by the CircuitJS engine. It provides all the necessary information about the components, their connections, and their electrical properties, enabling analysis such as transient response, DC/AC sweep, or frequency domain behavior.

The text file serves as input to a circuit simulation engine, which then performs numerical computations to evaluate how the circuit behaves under specified conditions.

For each component, this text file contains the information about its name, the nodes it is connected to on either side and the component's parameters or model (if necessary).

After the JSON file with abundant information about the components, their values, and the nodes they are connected to is generated, the next step is to convert said JSON file into a CircuitJS-compatible file (a `.txt` file which follows the appropriate conventions).

4.1.8 Digital Schematic Generation

The JSON file built by integrating all the information about the hand-drawn circuit is to be manipulated through different JavaScript libraries to generate a digital, editable schematic of the circuit. As briefly discussed in the Requirements section, react-draggable is used to map each component in JSON to a visual block (a symbol for the component as given by the KiCad symbol library). For each component, a graphical symbol needs to be rendered at a certain position and connection lines need to be drawn between the terminals and node positions. A rough pseudocode for the same is as follows:

4.1.9 Backend Development

The backend has been constructed using Next.js and FastAPI, functioning as the core middleware responsible for routing, and processing the system's Machine Learning (ML) pipeline and user interactions. All the endpoints are based on REST API which ensures the frontend and backend can communicate statelessly.

When the user uploads a circuit image through the UI, it reaches the upload endpoint (say /upload) which first saves it to temporary storage (may be local storage of browser). From there, a chain of processes which are involved in the project pipeline are executed.

Algorithm 1 Frontend Circuit Rendering

Require: Circuit JSON description J
Ensure: Rendered circuit schematic
  for all component $c \in J$.components **do**
    Draw schematic symbol of type c.type at position c.position
    Attach text label c.value near component position
  end for
  for all connection $k \in J$.connections **do**
    Retrieve terminal position from k.from and terminal index
    Retrieve node position from k.node
    Draw wire between terminal and node positions
  end for
  return Rendered schematic

The preprocessing steps, for both component detection and text recognition are first executed, and then, in a sequential fashion, the models as well. The tasks like deleting components, adding wires, changing values, resizing, rotating components, etc. are handled by CircuitJS and the role of backend in this case is just to be the facilitator for this, for communication and temporary storage.

4.1.10 LLM Integration

Using the publicly available Deepseek v3.1 API, the project is elevated from simple circuit manipulation tool to one capable of auto-analysis. The LLM has context about the uploaded image as well as the associated netlist or CircuitJS text file using which it can produce accurate responses to user's queries (prompts) related to their circuit. It can expain the reason for placement of a particular circuit component or why a particular value has been used, and much more.

4.2 Line Detection

To address the limitations of the junction-based wire detection algorithm which used only horizontal and vertical classification, Multi-Head BFS-based wire detection and generation algorithm was built. However, both approaches were extensively used and tested, and hence both have been explained in upcoming sections.

4.2.1 Line Detection via Horizontal and Vertical Classification

For this approach, the junction points detected by the component detection model (YOLOv7 at the end, but also other YOLO models during testing) are needed first. The coordinates given by YOLO falls under the range $[0, 1]^2$, so they need to be denormalized into pixel space using $\mathcal{N}^{-1}(x_n, y_n) = (W \cdot x_n,\ H \cdot y_n)$. Then, junction pairs are formed for processing. Every junction is paired up with every other junction. For a given pair $(\mathbf{j}_i, \mathbf{j}_k)$, the displacement between them and angular orientation is needed. For this, we

calculate the displacement vector using $\mathbf{d} = \mathbf{j}_k - \mathbf{j}_i$ and the angular orientation $\tilde{\theta}$ using arctangent.

Using $\tilde{\theta}$, the junction pairs are classified with 18° tolerance.

- **Horizontal:** $\tilde{\theta} \in [0°, 18°] \cup [162°, 198°] \cup [342°, 360°]$
- **Vertical:** $\tilde{\theta} \in [72°, 108°] \cup [252°, 288°]$
- **Diagonal (rejected):** remaining angles

Those pairs which are not rejected (i.e those with diagonal lines assumed between them) are projected onto perfectly horizontal or vertical lines. This is done since hand-drawn wire lines may be slightly jagged or imperfect in other ways. These lines are then snapped to a 16-pixel grid by consolidating overlapping segments which can be considered collinear (provided that they have the same angular orientation and fall within the grid tolerance). To determine where wires are actually connected to the components, intersection of each bounding box and any horizontal or vertical wire it touches is calculated.

In this approach, the wire connections are simply guessed based on the junction points, without taking into account the actual wire paths in the circuit. Those junction pairs which are completely unrelated but are axis-aligned may end up being connected. All issues in this step are inherited by terminal detection step as well, eventually leading to completely inaccurate results.

4.2.2 Line Detection via Path Finding

Instead of just guessing wire connectivity based on junction points, this method skeletonizes the entire circuit and then uses Multi-Head BFS to find its way through the circuit graph using endpoints as nodes.

Endpoint Detection: Endpoint detection is again, a task of the component detection model which generates bounding boxes for each component. Junction points have a single endpoint. The centre of the bounding box (x_c, y_c) is snapped to the skeleton pixel that sits the closest to it, within the search region (with a margin of 8 pixels). Components, however, need two endpoints. All the skeleton pixels that lie directly on the bounding box (on either side or even on top of the component) or in its close vicinity are scanned. The two pixels with the greatest separation between them is selected as the two (or more, for components like transistors or IC) endpoints of the component.

Graph Construction via Multi-Head BFS: The detected endpoints act as nodes and the skeleton as edges help build a circuit graph. A disk with radius r is formed around each node and a list (`node_id_map`) that associates the node with each pixel that is

owned by said node is made. This map carries -1 if no node owns a particular pixel. BFS is run from each node starting right from the particular pixels at the boundary of the aforementioned disk with a queue being formed, as is typical of a BFS algorithm. This BFS queue moves along the skeletonized circuit through 8-connectivity. The moment the BFS traversal encounters a pixel owned by some other node j, the edge (i, j), with i being the starting node, is logged and that particular branch stops. After BFS is run from all the end points, the graphs gets constructed and self-edges are pruned. These are edges connecting two endpoints of the same bounding box, and stripping them out guarantees that only inter-object connections survive.

Advantages over horizontal/vertical classification: Because path-finding follows the actual wire paths visible in the image, phantom connections vanish. Wire orientation ceases to matter altogether; horizontal and vertical wires get handled just the same as diagonal ones. Wire detection and terminal detection collapse into one graph-construction step. Scaling is linear with skeleton pixel count instead of quadratic with junction count. For all of these reasons, the final deployed system runs on this approach.

4.2.3 Crossover Dissolution Algorithm

Hand-drawn circuit diagrams contain two visually similar yet electrically opposite situations i.e, a *crossover*, where two wires pass over each other without any electrical contact, and a *junction*, where wires genuinely meet. Getting the distinction wrong ruins circuit reconstruction entirely. YOLOv7 flags crossovers as their own class, with one endpoint. Multi-Head BFS then flags this one node as being connected to four neighboring endpoints associated with two wires (or wire segments) intersecting without being connected at that node. The job of the dissolution algorithm is to correctly identify which two wire segments are actually electrically connected.

A crossover, by definition, has four neighbors. This means an endpoint with less than four neighbors is simply a junction and is handled as such. For a detected crossover at a position $\mathbf{p}$, and its four neighbors as represented by $\mathbf{n}_k$, a unit direction vector toward each neighbor is given as:

$$\hat{\mathbf{d}}_k = \frac{\mathbf{n}_k - \mathbf{p}}{\|\mathbf{n}_k - \mathbf{p}\|} \tag{4-11}$$

All possible pairs of neighbors (for four neighbors, two pairs in three ways) are tested. The sum of dot product is calculated for each pairing (that may be situated at opposite ends of a crossover) $\{(\mathbf{n}_a, \mathbf{n}_b),\ (\mathbf{n}_c, \mathbf{n}_d)\}$.

$$S = \hat{\mathbf{d}}_a \cdot \hat{\mathbf{d}}_b + \hat{\mathbf{d}}_c \cdot \hat{\mathbf{d}}_d \tag{4-12}$$

The pairs resulting in the most negative value are considered to be situated the most further away (opposite ends of the crossover, in this case) from one another. For confirmation, a threshold for each dot product in the most negative value of the sum $\hat{\mathbf{d}}_a \cdot \hat{\mathbf{d}}_b < -0.7$ must be cleared. This ensures the dot product pairs are roughly 45° apart from one another, through the crossover center.

After this check is successful, the crossover node is deleted from the adjacency graph to be replaced by two new edges, with each linking one pair of neighbors (from the selected two pairs) that are opposite to each other, directly. The crossover i.e, a four-neighbor junction now gets replaced by two wires that cross each other while being electrically disconnected. If the check is failed, the node that was initially flagged as a crossover is reverted back to a junction, with all four wire segments having electrical connections among them.

4.3 Use-Case Diagram

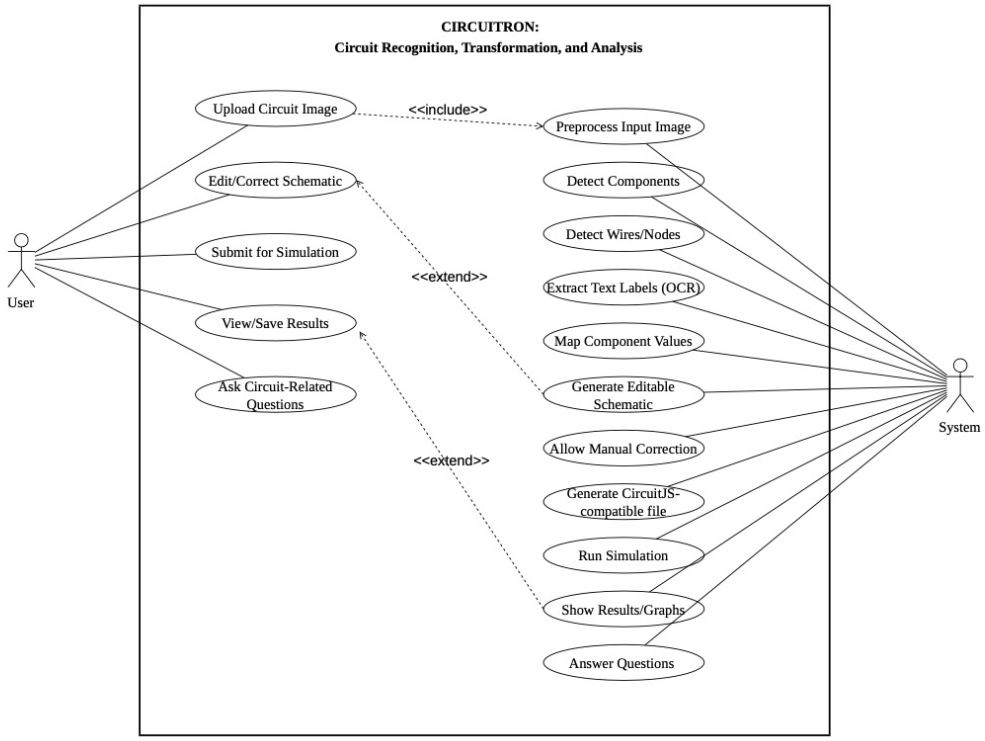


Figure 4-5 Use-Case Diagram for CIRCUITRON System

Figure 4-5 is the use-case diagram for the CIRCUITRON system which is a diagrammatical representation of all that a typical user and the system perform in the project pipeline. The actors in this use-case diagram are the user (student or instructor) and the system. The user uploads the circuit image, edits schematic, submits

for simulation and save results. The user may also ask circuit-related questions to the integrated LLM in the system.

When the user uploads a circuit sketch, the system pre-processes this image, fires up the component detection and text recognition models for recognizing circuit elements along with their values (and units) and these are mapped as appropriate. The system then generates an editable schematic. If the user submits for simulation, it runs the simulation to generate graphs and results. The LLM also answers user queries with in-built context.

The includes relationship between some use-cases signifies that a particular task must be completed before the task pointed by the arrow can be started. In contrast, the extends relationship signifies optional tasks that is not necessary for another task to be started, but may "extend" the functionality of a particular use-case.

4.4 Performance Metrics Expectation

It is important, in any ML-related project, to establish an expectation of performance by the models, which in turn, may be used to consider the model to be "good-enough" and halt further training. The choice of performance metric depends on the kind of model (for instance, it will be different for object detection and text recognition). The choice of performance metric and target or expected value is listed in Table 4-1.

Table 4-1 Expected Performance Metric Values for Individual Modules

Pipeline Stage	**Performance Metric**	**Target Value/Expectation**
Component Detection	mAP@0.5	0.95
Text Recognition	Character Accuracy	0.65

4.5 System Flowchart

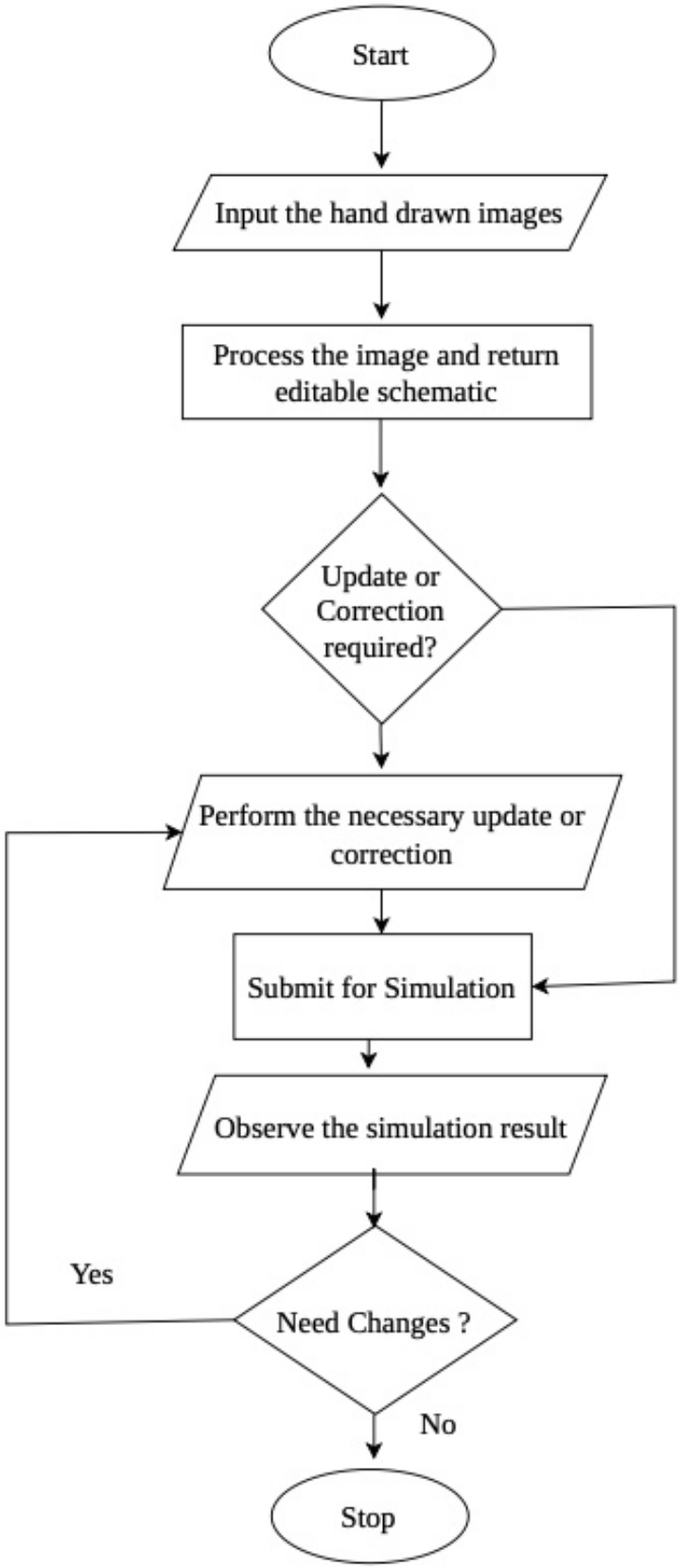


Figure 4-6 System Flowchart

The key architectural modules involved in the working of the system have been demonstrated by the block diagram in Figure 4-1. Now, the flowchart in Figure 4-6 demonstrates the motions a user needs to go through to use the final product. Once the image of a hand-drawn circuit is uploaded, the system processes the image (internally using the techniques explained in the previous sections) and returns an editable schematic. Any necessary update or correction may be performed if necessary and then the schematic can be submitted for simulation and analysis. Relevant graphs are returned after analysis.

4.6 Dataset Analysis

In order to develop the system using the system for automated circuit diagram understanding, the Circuit Graph Handwritten Diagrams (CGHD) (Circuit Graph Handwritten Diagrams) dataset is used. This dataset provides a comprehensive suite of ground truth annotations tailored to facilitate key computer vision tasks such as object detection, instance segmentation, and text recognition. While additional datasets (such as circuitVQA, Digitize-HCD datasets) may be used to fine-tune the models in the future to achieve even higher values for performance metrics, as of yet, only the CGHD dataset has been used.

4.6.1 Dataset Overview

The CGHD dataset comprises 4.99 GB of annotated data spanning over 3,000 raw images. The data is separated into folders named drafters, going from -1 through 31, and each drafter folder contains hand-sketches of 12 unique circuit designs, with each circuit drawn twice and captured under four different conditions (varying angles, lighting, and minor distortions), resulting in up to 96 images per drafter. As is convention, the dataset is to be split into (70-10-20)% of the entire data for training, validation and testing respectively.

Table 4-2 Summary of Data in the CGHD Dataset

Items	**Number**
Distinct Circuits	372
Raw Images	3,269
Bounding Box Annotations	248,020
Text String Annotations	85,417 (93.74% completeness)
Character Types	98
Object Classes	61

Every image is rigorously annotated and the annotations provided per image include:

- **Bounding Boxes (PASCAL VOC format)** for 61 electrical component classes, including resistors, capacitors, transistors, operational amplifiers, and textual elements.
- **Text Labels** that identify a component and its value, with unit (e.g., "$10k\Omega$", "$C1$", "V").
- **Instance Segmentation Polygons** in the LabelMe JSON format that indicate outlines of the object in the pixel-level.
- **Binary Segmentation Maps** that distinguish background and foreground (e.g., circuit components in the foreground, paper lines in the background).
- **Partial Netlist Files** (in `.asc` format) that are SPICE compatible, for testing circuit construction for some circuits.

The text annotation coverage is extensive, with labels ranging from single characters (like "R" or "$R1$") to full descriptors (like "Resistor", "Variable Resistor", etc.).

4.6.2 Class-wise Data Statistics

Table 4-3 Frequency of Each Class with its Code in the CGHD Dataset

Code	Class Name	Count	Code	Class Name	Count
1	text	91,122	32	opamp.schmitt	16
2	junction	79,202	33	optocoupler	162
3	crossover	10,180	34	ic	1,506
4	terminal	11,253	35	ic.ne555	262
5	gnd	5,501	36	ic.v_regulator	362
6	vss	1,354	37	xor	162
7	voltage.dc	335	38	and	860
8	voltage.ac	292	39	or	440
9	voltage.battery	765	40	not	496
10	resistor	14,400	41	nand	228
11	resistor.adj	1,052	42	nor	59
12	resistor.photo	65	43	probe	184
13	cap.unpol	5,841	44	probe.curr	72
14	cap.pol	2,227	45	probe.volt	32
15	cap.adj	88	46	switch	2,184
16	inductor	1,976	47	relay	89
17	inductor.ferrite	120	48	socket	184
18	inductor.coupled	10	49	fuse	226
19	transformer	483	50	speaker	410
20	diode	3,890	51	motor	108
21	diode.led	1,212	52	lamp	301
22	diode.thyrector	16	53	microphone	72
23	diode.zener	609	54	antenna	24
24	diac	40	55	crystal	112
25	triac	98	56	mechanical	32
26	thyristor	415	57	magnetic	364
27	varistor	184	58	optical	127
28	transistor.bjt	3,678	59	block	0
29	transistor.fet	977	60	explanatory	610
30	transistor.photo	177	61	unknown	32
31	opamp	752			

5 IMPLEMENTATION DETAILS

The implementation of the system to convert hand-drawn circuit diagrams into simulation-ready netlists and digital schematics follows a modular approach, aligning with the pipeline described in the architecture and methodology.

5.1 Dataset Manipulation and Preprocessing

5.1.1 Conversion of Annotation Formats

To accommodate the multiple models in the pipeline (YOLOv7 for component detection and OCR for value extraction), annotation files provided in the CGHD dataset in Pascal VOC (XML) format were converted into compatible formats. Likewise, for EasyOCR-inspired custom-trained OCR, all text-sections needed to be cropped out from the image with corresponding labels (ground truth) and then fed to the fine-tuned model.

In Pascal VOC format, the information regarding the circuit and its components are stored in the following format as shown by the example below:

```
<annotation>
    <folder>images</folder>
    <filename>C1_D1_P2.jpg</filename>
    <path>./drafter_1/images/C1_D1_P2.jpg</path>
    <source>
        <database>CGHD</database>
    </source>
    <size>
        <width>3024</width>
        <height>4032</height>
        <depth>3</depth>
    </size>
    <segmented>0</segmented>
    <object>
        <name>voltage.dc</name>
        <pose>Unspecified</pose>
        <truncated>0</truncated>
        <difficult>0</difficult>
        <bndbox>
            <xmin>327</xmin>
            <ymin>546</ymin>
            <xmax>604</xmax>
            <ymax>838</ymax>
```

```
            <rotation>10</rotation>
        </bndbox>
    </object>
</annotation>
```

For YOLO-based detection, XML annotations were converted to YOLO format (`.txt`) using a custom Python script. Each `.txt` file contains the class index and normalized bounding box coordinates in the format:

```
<class_id> <x_center> <y_center> <width> <height>
```

The pseudocode for the VOC to YOLO annotation conversion is as follows:

Algorithm 2 VOC to YOLO Annotation Conversion

Require: XML annotation file X, class map C, image width W, image height H
Ensure: List of YOLO formatted annotation lines Y

```
Parse XML file X into tree structure
Y ← ∅
for all object nodes o in XML root do
    clsname ← class name of o
    if clsname ∉ C then
        continue
    end if
    clsid ← C[clsname]
    Extract bounding box (x_min, y_min, x_max, y_max) from o
    Convert VOC coordinates to normalized YOLO format
    x_center ← (x_min + x_max) / (2W)
    y_center ← (y_min + y_max) / (2H)
    w ← (x_max − x_min) / W
    h ← (y_max − y_min) / H
    Append formatted string (x_center, y_center, w, h) to Y
end for
return Y
```

The calculations run for this conversion is as follows:

$$x_{\text{center}} = \frac{327 + 604}{2 \times 3024} = \frac{465.5}{3024} = 0.1539 \tag{5-1}$$

$$y_{\text{center}} = \frac{546 + 838}{2 \times 4032} = \frac{692}{4032} = 0.1716 \tag{5-2}$$

$$w = \frac{604 - 327}{3024} = \frac{277}{3024} = 0.0916 \tag{5-3}$$

$$h = \frac{838 - 546}{4032} = \frac{292}{4032} = 0.0724 \tag{5-4}$$

This gives the information required by YOLO in the appropriate format. It is convention to save this new annotation file in the same directory as the image.

Likewise, for OCR compatibility, the same Pascal VOC annotations were converted to Lightning Memory-Mapped Database (LMDB) format. LMDB is a high-speed key-value store where each key corresponds to an index and each value contains an image-text pair (serialized).

The pseudocode for the conversion of Pascal VOC format to OCR-compatible LMDB file format is as follows:

Algorithm 3 Image Cropping and LMDB Dataset Creation

Require: Input image file I, crop coordinates $(y_{\min}, y_{\max}, x_{\min}, x_{\max})$, target size (W, H)
Ensure: LMDB database containing image-label pairs
  Load image I into memory
  Crop image using coordinates $[y_{\min} : y_{\max}, x_{\min} : x_{\max}]$
  Resize cropped image to size (W, H)
  Convert cropped image to byte representation
  Convert image array to PIL image
  Initialize in-memory byte buffer
  Save image to buffer in JPEG format
  Extract byte stream from buffer
  Store image and label in LMDB
  Open LMDB environment with predefined map size
  Begin write transaction
  Store image bytes with key `image-00001`
  Store corresponding label with key `label-00001`
  Store total sample count with key `num-samples`
  Commit transaction
  return LMDB environment

After this script is run in its entirety, a file in LMDB format, as is expected by OCR, is received as shown in the example below:

```
Key: b'image-00001'
Value: b'\xff\xd8\xff\xe0\x00\x10JFIF...'

Key: b'label-00001'
Value: b'voltage.dc'

Key: b'num-samples'
Value: b'000001'
```

After necessary conversion, the dataset was ready for being plugged into the required models for fine-tuning them. But first, some experimentation was done with different forms of image augmentation techniques.

5.1.2 Image Augmentation

Since the dataset contains handwritten circuit diagrams captured under variable lighting and orientations, several augmentation techniques were applied to improve model generalization including Contrast Enhancement (global) and Adaptive Brightness Adjustment by factors of 0.8 and 1.2. Furthermore, geometric augmentation techniques, like spatial flipping (both horizontally and vertically) and rotation (e.g., 90° or random rotation) proved to be unwise operations for this project. For example, if a "C" is rotated by 90°, the "C" now resembles a "U", and the same applies to flipping ("M" might look like "W", for example). Rather than increasing accuracy, it was realized that using rotation or flipping would simply confuse the model.

5.1.3 Fine-tuning YOLOv5 and YOLOv7 for Component Detection

YOLOv5 was initially used for benchmarking, and later upgraded to YOLOv7 for better accuracy and inference speed (comparison shown in the Results section). Training and fine-tuning were done using the CGHD dataset after converting into YOLO-compatible format.

The training parameters employed are as follows:

- **Number of classes:** 61
- **Training split:** 70% training, 10% validation, 20% testing
- **Epochs:** 200 (for YOLOv5), 100 (for YOLOv7)
- **Image size:** 416×416 pixels
- **Batch size:** 16 (for YOLOv5), 32 (for YOLOv7)
- **Optimizer:** Stochastic Gradient Descent (SGD) (Stochastic Gradient Descent)
- **Learning rate:** 0.01 (initial)
- **Augmentations used in training:** Mosaic Augmentation, Hue, Saturation, Value (HSV) Augmentation, Random rotating/flipping

5.1.4 Class Consolidation and Retraining

Satisfactory results were not obtained through the initial 61-class approach. To improve the performance, consolidation of 61 classes into 15 was done, and the models were retrained on this new taxonomy.

Table 5-1 Consolidation of Original Classes into Unified YOLO Classes

Class Code	Class Name	Consolidated Classes
0	capacitor	capacitor.adjustable, capacitor.polarized, capacitor.unpolarized
1	crossover	crossover
2	diode	diode, diode.light_emitting, diode.thyrector, diode.zener
3	gnd	gnd
4	inductor	inductor, inductor.coupled, inductor.ferrite
5	integrated_circuit	integrated_circuit, integrated_circuit.ne555, integrated_circuit.voltage_regulator
6	junction	junction
7	operational_amplifier	operational_amplifier, operational_amplifier.schmitt_trigger
8	resistor	resistor, resistor.adjustable, resistor.photo
9	switch	switch
10	terminal	terminal
11	text	text
12	transistor	transistor.bjt, transistor.fet, transistor.photo
13	voltage	voltage.ac, voltage.battery, voltage.dc
14	vss	vss

A merge mapping was applied to the original annotations, grouping visually similar subclasses (e.g., capacitor.polarized and capacitor.unpolarized into capacitor) as shown in the Table 5-1. This consolidation is done not only considering visual similarity but also based on the usage i.e, the resulting 15 classes make up the majority of basic-level circuits Four YOLO architectures were then trained on this 15-class dataset for benchmarking:

- **YOLOv7 (retrained):** 100 epochs, batch size 16, image size 640×640
- **YOLOv8:** 100 epochs, batch size 16, image size 640×640
- **YOLOv11:** 100 epochs, batch size 16, image size 640×640
- **YOLOv26:** 100 epochs, batch size 16, image size 640×640

All models were trained on identical data splits with the same augmentation pipeline, confidence threshold of 0.25, and IoU threshold of 0.45.

5.1.5 Implementation of Custom Text Recognition (OCR)

To extract textual information such as resistor values, capacitor labels, and other component annotations from the circuit diagrams, a CRNN-based custom OCR model was trained, and implemented. The input image in cropped form (cropped for each text component) was passed through this model after training, which returned the content of

each detected text element. The position of said text element was found using bounding boxes from YOLOv7. This allowed us to localize and read numerical values or labels directly from the circuit.

Three different modules, each for feature extraction, sequence modeling, and prediction were trained based on the DTRB framework. Initially, simply EasyOCR was also attempted; however, this didn't yield usable results, ultimately requiring training from scratch. The training, which was done for 27 epochs, had early stopping as it's accuracy per epoch failed to achieve even a 0.01% increase within 5 epochs. Other training details include:

- **Training epochs:** 27 (with early stopping)
- **Image Height:** 64px (fixed for all croppings)
- **Aspect Ratio:** 4:1 (allowing for variable width)
- **Learning Rate:** 0.01 (initial)

5.1.6 Fine-tuning TrOCR for Text Recognition

Following the initial custom CRNN approach, a second OCR model was implemented using Microsoft's TrOCR-small-printed architecture. Unlike the CRNN model which was trained from scratch, TrOCR was fine-tuned from pre-trained weights, leveraging transfer learning from large-scale printed text datasets.

The fine-tuning was performed on cleaned text crops extracted from the CGHD dataset. All text crops from a single image were processed in one forward pass, and per-token log-probability was averaged across the generated sequence with following details:

- **Base model:** Microsoft TrOCR-small-printed (pre-trained)
- **Fine-tuning epochs:** 3
- **Checkpoint:** Epoch 2 (best validation performance)
- **Inference precision:** float16 (half-precision for GPU acceleration)
- **Maximum output tokens:** 16 (sufficient for values like "10kΩ", "100μF")

The TrOCR model is integrated into the pipeline through a singleton service that lazy-loads the processor and model weights when it is first invoked.

5.1.7 Line Detection Algorithm Implementation

Any sort of line detection algorithm in service of detecting wires in a circuit diagram exploit the regularity inherent to standard circuit diagrams. This junction-based method

takes the junction points that YOLOv7 has already picked up and treats them as anchor nodes for rebuilding the wiring layout through geometry.

Algorithm 4 Wire Classification and Alignment

Require: Set of junctions J, angular tolerance $\omega_{\text{tol}} = 18°$, grid size $g = 16$
Ensure: Set of raw wire segments S

$S \leftarrow \emptyset$
for all pairs $(j_a, j_b) \in \text{Combinations}(J, 2)$ **do**
  $\Delta x \leftarrow j_b.x - j_a.x$
  $\Delta y \leftarrow j_b.y - j_a.y$
  $\theta \leftarrow \text{atan2}(\Delta y, \Delta x)$
  if $|\theta| \le \omega_{\text{tol}} \vee |\theta - 180°| \le \omega_{\text{tol}}$ **then**
    Horizontal alignment check
    $y_{\text{avg}} \leftarrow \frac{j_a.y + j_b.y}{2}$
    $y_{\text{snap}} \leftarrow \text{Round}\left(\frac{y_{\text{avg}}}{g}\right) \times g$
    $S \leftarrow S \cup \{(H, y_{\text{snap}}, \min(j_a.x, j_b.x), \max(j_a.x, j_b.x))\}$
  else if $|\theta - 90°| \le \omega_{\text{tol}} \vee |\theta - 270°| \le \omega_{\text{tol}}$ **then**
    Vertical alignment check
    $x_{\text{avg}} \leftarrow \frac{j_a.x + j_b.x}{2}$
    $x_{\text{snap}} \leftarrow \text{Round}\left(\frac{x_{\text{avg}}}{g}\right) \times g$
    $S \leftarrow S \cup \{(V, x_{\text{snap}}, \min(j_a.y, j_b.y), \max(j_a.y, j_b.y))\}$
  end if
end for
return S

Pairwise comparison has a predictable side effect: redundant overlapping segments pile up for the same wire. Multiple separate segments are seen for multiple junctions if they are sitting on the same line, but in reality, there exists only one continuous wire. To address this, collinear segment merging is needed.

After classifying the segments as horizontal or vertical, the overlapping or neighboring line segments get merged together. During each merge, the axis coordinate of whichever segment is longer takes priority, since a longer stroke is a more trustworthy indicator of where the wire actually runs.

The wire–component intersection detection algorithm pinpoints where detected wire segments meet component bounding boxes. YOLOv7 annotations supply the component regions, which then get translated into pixel coordinates. Every horizontal or vertical wire is checked against each component box's edges for geometric intersection. Those intersection points pin down how the circuit is actually wired together, and the frontend can optionally render them for visual verification.

Algorithm 5 Collinear Segment Merging

Require: List of segments L, gap tolerance ω
Ensure: List of merged segments M
  Sort L by primary axis, then by starting coordinate
  $M \leftarrow \emptyset$
  curr $\leftarrow L[0]$
  for all next $\in L[1 \ldots N]$ **do**
    if $|\text{curr.axis} - \text{next.axis}| \leq \omega \wedge \text{next.start} \leq \text{curr.end} + \omega$ **then**
      curr.start $\leftarrow \min(\text{curr.start}, \text{next.start})$
      curr.end $\leftarrow \max(\text{curr.end}, \text{next.end})$
      if Length(next) > Length(curr) **then**
        Preserve axis of the longer segment
        curr.axis $\leftarrow$ next.axis
      end if
    else
      $M \leftarrow M \cup \{\text{curr}\}$
      curr $\leftarrow$ next
    end if
  end for
  $M \leftarrow M \cup \{\text{curr}\}$
  return M

Algorithm 6 Wire–Component Intersection Detection

Require: Wire segments W, YOLO label file L, image width I_w, image height I_h
Ensure: Set of intersection points P
  Parse label file L
  Convert normalized bounding boxes to pixel coordinates
  Discard non-component classes
  Store component boxes B
  Detect intersections between wires and components
  $P \leftarrow \emptyset$
  for all wire $w \in W$ **do**
    for all box $b \in B$ **do**
      if w is horizontal and crosses left or right edge of b **then**
        Add intersection point to P
      else if w is vertical and crosses top or bottom edge of b **then**
        Add intersection point to P
      end if
    end for
  end for
  Draw wires, component boxes, and intersection points on image
  Save output visualization
  return P

5.2 Line Detection via Path Finding

The algorithm behind the path-finding line detection pipeline first splits the work into a handful of stages. First, a spatial map of node regions is assembled. Multi-head BFS traversal then crawls through skeleton pixels to uncover which nodes connect to which. Spurious self-edges get pruned in a final cleanup pass.

Building a node identification map kicks off the whole process. This map contains information regarding if a particular pixel falls inside a node (node's circuilar disk region) or not. Algorithm 7 details the construction of this node id map.

Algorithm 7 Build Node ID Map

Require: Skeleton image S of size $H \times W$, list of nodes $\mathcal{N} = \{(x_0, y_0), \ldots, (x_{n-1}, y_{n-1})\}$, radius r
Ensure: Node ID map M of size $H \times W$

$M \leftarrow$ array of size $H \times W$ initialized to -1
$\mathcal{D} \leftarrow \text{DiskMask}(r)$
for $i \leftarrow 0$ **to** $n - 1$ **do**
    $(n_x, n_y) \leftarrow \mathcal{N}[i]$
    for each $(dy, dx) \in \mathcal{D}$ **do**
        $(p_y, p_x) \leftarrow (n_y + dy, n_x + dx)$
        if $0 \leq p_y < H$ **and** $0 \leq p_x < W$ **then**
            $M[p_y][p_x] \leftarrow i$
        end if
    end for
end for
return M

Once the node ID map is constructed, the algorithm performs a breadth-first search from each node to discover its neighbors along the skeleton. The BFS starts from the boundary of each node's disk region and traverses skeleton pixels using 8-connectivity. When this BFS traversal meets a pixel belonging to a different node, that node is recorded as a neighbor which terminates the branch. This ensures that each skeleton path is followed until it reaches another endpoint. Algorithm 8 describes this procedure.

Algorithm 8 BFS Neighbors for Node

Require: Node index i, nodes $\mathcal{N}$, skeleton S, node ID map M, radius r
Ensure: Set of neighbor node indices $\mathcal{A}_i$

$(n_x, n_y) \leftarrow \mathcal{N}[i]$
$\mathcal{A}_i \leftarrow \emptyset$
$V \leftarrow$ boolean array of size $H \times W$ initialized to `false`
$\mathcal{D} \leftarrow \text{DiskMask}(r)$
for each $(dy, dx) \in \mathcal{D}$ **do**
    $(p_y, p_x) \leftarrow (n_y + dy, n_x + dx)$
    if $0 \le p_y < H$ **and** $0 \le p_x < W$ **then**
        $V[p_y][p_x] \leftarrow$ `true`
    end if
end for
$Q \leftarrow$ empty queue
for each $(dy, dx) \in \mathcal{D}$ **do**
    $(p_y, p_x) \leftarrow (n_y + dy, n_x + dx)$
    for each $(n'_y, n'_x) \in \text{Neighbors8}(p_y, p_x)$ **do**
        if $\neg V[n'_y][n'_x]$ **and** $S[n'_y][n'_x] > 0$ **then**
            $V[n'_y][n'_x] \leftarrow$ `true`
            $Q.\text{enqueue}((n'_y, n'_x))$
        end if
    end for
end for
while Q is not empty **do**
    $(c_y, c_x) \leftarrow Q.\text{dequeue}()$
    $j \leftarrow M[c_y][c_x]$
    if $j \neq -1$ **and** $j \neq i$ **then**
        $\mathcal{A}_i \leftarrow \mathcal{A}_i \cup \{j\}$
        continue
    end if
    for each $(n'_y, n'_x) \in \text{Neighbors8}(c_y, c_x)$ **do**
        if $\neg V[n'_y][n'_x]$ **and** $S[n'_y][n'_x] > 0$ **then**
            $V[n'_y][n'_x] \leftarrow$ `true`
            $Q.\text{enqueue}((n'_y, n'_x))$
        end if
    end for
end while
return $\mathcal{A}_i$

The final step removes spurious connections between endpoints that belong to the same detected object. Since line segment bounding boxes produce two endpoints each, the BFS may incorrectly connect these as neighbors via the skeleton segment within the bounding box. By maintaining a list of endpoint pairs from the same object, these self-edges are explicitly removed from the adjacency graph. Algorithm 9 implements this filtering.

Algorithm 9 Remove Self-Edges (Same-Object Connections)

Require: Adjacency $\mathcal{G}$, endpoint pairs $\mathcal{P} = \{(p_1^{(k)}, p_2^{(k)})\}_{k=1}^{m}$, nodes $\mathcal{N}$
Ensure: Filtered adjacency $\mathcal{G}'$

$\mathcal{G}' \leftarrow \mathcal{G}$
for each $(p_1, p_2) \in \mathcal{P}$ **do**
    $i \leftarrow \text{FindIndex}(p_1, \mathcal{N})$
    $j \leftarrow \text{FindIndex}(p_2, \mathcal{N})$
    if $i \neq -1$ **and** $j \neq -1$ **then**
        $\mathcal{G}'[i] \leftarrow \mathcal{G}'[i] \setminus \{j\}$
        $\mathcal{G}'[j] \leftarrow \mathcal{G}'[j] \setminus \{i\}$
    end if
end for
return $\mathcal{G}'$

5.2.1 Complexity of Implemented Algorithm

This section analyzes the computational complexity of the line detection pipeline. Let H, W denote the image dimensions, n the number of nodes (endpoints), r the disk radius for node regions, and P the number of skeleton pixels (typically $P \ll H \times W$).

Table 5-2 Time complexity breakdown for Multi-Head BFS algorithm

Operation	**Complexity**	**Description**
Node ID Map Construction	$O(n \cdot r^2)$	Each node marks πr^2 pixels
Single Node BFS	$O(P)$	Each skeleton pixel visited at most once
Complete Adjacency Build	$O(n \cdot P)$	BFS from each of n nodes
Self-Edge Removal	$O(m)$	m = number of endpoint pairs
Total	$O(n \cdot r^2 + n \cdot P)$	Dominated by BFS traversals

Table 5-2 summarizes the time complexity of each operation in the pipeline.

Table 5-3 Space complexity breakdown for Multi-Head BFS algorithm

Data Structure	**Space**	**Description**
Skeleton Image S	$O(H \times W)$	Binary image storage
Node ID Map M	$O(H \times W)$	Integer array for node mapping
Visited Array V	$O(H \times W)$	Per-BFS boolean array
BFS Queue Q	$O(P)$	Maximum skeleton pixels in queue
Adjacency $\mathcal{G}$	$O(n + E)$	E = number of edges
Disk Offsets $\mathcal{D}$	$O(r^2)$	Precomputed disk mask
Total	$O(H \times W + n + E)$	Dominated by image arrays

Table 5-3 details the memory requirements of the algorithm.

The visited array can be reused across BFS calls with a reset, or alternatively, a hash set can be used for sparse skeletons to reduce visited storage to $O(P)$. For typical skeleton images where $P = O(H + W)$ (linear in image perimeter), the overall complexity is:

$$T(n, H, W, r) = O(n \cdot r^2 + n \cdot (H + W)) = O(n \cdot (r^2 + H + W))$$

In the real world, this approach can be considered efficient because it scales with the number of nodes, but it is bound by the physical resolution of the image sensor.

5.2.2 Crossover Dissolution

After the adjacency graph is constructed by Multi-Head BFS, crossover nodes must be resolved into their true topology. A crossover represents two wires crossing without connecting, and is initially represented as a single graph node with four neighbors. These crossover nodes are replaced by two direct connections, as described by Algorithm 10.

Algorithm 10 Crossover Dissolution Algorithm

Require: Adjacency graph $\mathcal{G}$, node positions $\mathbf{p}$, detection results $\mathcal{R}$
Ensure: Modified graph $\mathcal{G}$ with crossovers dissolved

for each detection $r \in \mathcal{R}$ with $r.\text{cls} = 1$ **do**
  $a \leftarrow$ node index of r's endpoint in $\mathbf{p}$
  if $a = -1$ **or** $|\mathcal{G}[a]| \neq 4$ **then**
    continue
  end if
  $\{n_1, n_2, n_3, n_4\} \leftarrow \mathcal{G}[a]$
  for each neighbor n_k **do**
    $\hat{\mathbf{d}}_k \leftarrow \dfrac{\mathbf{p}[n_k] - \mathbf{p}[a]}{\|\mathbf{p}[n_k] - \mathbf{p}[a]\|}$
  end for
  $S^* \leftarrow +\infty, \quad \Pi^* \leftarrow \varnothing$
  for each pairing $\Pi = \{(n_i, n_j), (n_k, n_l)\}$ of the 4 neighbors into 2 pairs **do**
    $S \leftarrow \hat{\mathbf{d}}_i \cdot \hat{\mathbf{d}}_j + \hat{\mathbf{d}}_k \cdot \hat{\mathbf{d}}_l$
    if $S < S^*$ **then**
      $S^* \leftarrow S, \quad \Pi^* \leftarrow \Pi$
    end if
  end for
  $\{(n_a, n_b), (n_c, n_d)\} \leftarrow \Pi^*$
  if $\hat{\mathbf{d}}_a \cdot \hat{\mathbf{d}}_b > -0.7$ **or** $\hat{\mathbf{d}}_c \cdot \hat{\mathbf{d}}_d > -0.7$ **then**
    continue
  end if
  for each neighbor $n_k \in \{n_a, n_b, n_c, n_d\}$ **do**
    $\mathcal{G}[n_k] \leftarrow \mathcal{G}[n_k] \setminus \{a\}$
  end for
  Remove a from $\mathcal{G}$
  Add edges (n_a, n_b) and (n_c, n_d) to $\mathcal{G}$
end for
return $\mathcal{G}$

5.3 Web App Implementation and Interaction

The frontend was built using React.js and Next.js alongside CircuitJS. To bring all the frontend features to life, such as resizing, moving, rotating, renaming, etc. of the circuit elements with proper integration with the backend, additional packages and libraries like react-draggable and the KiCad Symbol Library were used. The backend of the CIRCUITRON system is a bunch of Representational State Transfer (REST) Application Programming Interface (API) endpoints that handle all the functions of the system such as uploading the circuit sketch, edit schematics (using temporary storage for changes), generate schematic, correction of values of components, etc.

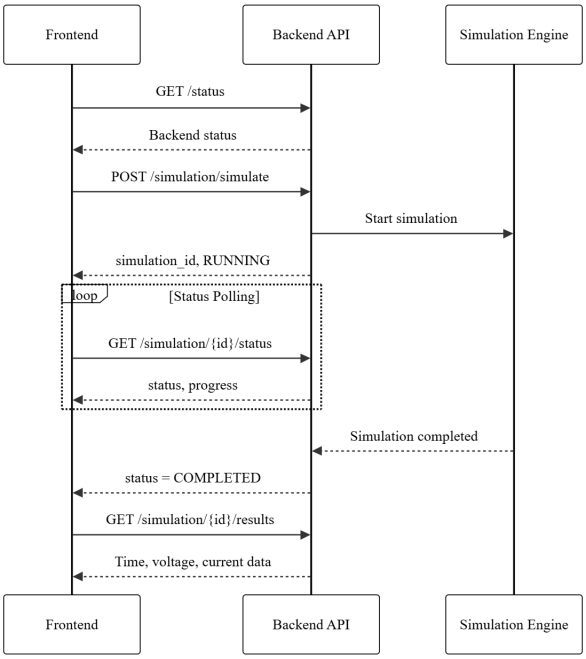


Figure 5-1 Interaction Diagram for the Web Application

The endpoints defaults to JSON for communication and uses HTTP status codes for error reporting. The edits stored in temporary storage are made permanent through regular polling between backend and frontend (initiated by frontend). The simulation is handled, primarily, using an open source browser-based electronic circuit simulator named CircuitJS. Internally, CircuitJS mathematically models the circuit in the form of matrices, simplifying computations.

5.4 Error Handling

TTo handle errors in this project, foremost the errors in the output of the individual modules need to be controlled and only then can the accumulated error be kept in check. The performance metrics tabulated in the previous section are to be strictly adhered to if there's any hope of generating the CircuitJS compatible text file (and digital schematic) with nominal errors.

For the reduction of false matches and OCR noise, post-processing techniques such as value format validation, confidence filtering, and text cleanup can be implemented. Value format validation ensures only strings that match known (and appropriate) electrical units, say $10k$, $1\mu F$, $5V$ are retained. Confidence filtering helps discard OCR results with low confidence scores, and text cleanup normalizes ambiguous symbols such as converting "O" to "0", "μ" to "u".

A basic pseudocode for sample value format validation is as follows:

Algorithm 11 Validation and Filtering of OCR Component Values

Require: OCR detected text entries T
Ensure: Filtered list of valid component values V
  Define regular expression patterns for resistance, capacitance, and voltage
  $V \leftarrow \emptyset$
  for all text entry $t \in T$ **do**
    if t.text matches any defined pattern **then**
      Append t to V
    end if
  end for
  return V

The list of components shown here is in no way meant to be exhaustive; rather it is to show how this pseudocode ensures that only valid electrical component values are used in the final component-to-value mapping and schematic generation.

To prevent propagating error (error from the output of YOLOv7 bleeding into OCR, for instance), a combined confidence threshold checking mechanism can be implemented which marks an item as uncertain if the confidence score from YOLOv7 is less than a certain value (e.g., 0.6) and the same from OCR is less than a certain value (e.g., 0.85). For more robustness, a human-in-the-loop mechanism can be implemented which prompts the user to manually approve or correct OCR-predicted values, fix connection errors, etc. This can be implemented to trigger when the confidence score is below the set threshold or when other error handling mechanisms such as value format validation fail. The pseudocode for this is as follows:

Algorithm 12 Confidence-Based Error Checking and Validation

Require: Module outputs O, confidence threshold ω, format validation function f
Ensure: Valid outputs V and uncertain outputs U
  $V \leftarrow \emptyset$
  $U \leftarrow \emptyset$
  for all item $o \in O$ **do**
    if o.confidence $\geq \omega \wedge f(o.\text{value})$ **then**
      Append o to V
    else
      Append o to U
    end if
  end for
  if $U \neq \emptyset$ **then**
    Trigger manual override
  end if
  return (V, U)

6 RESULTS AND ANALYSIS

6.1 Comparative Analysis of Object Detection using YOLOv5 and YOLOv7

6.1.1 Experimental Setup

A comparative study using YOLOv5 and YOLOv7 to detect and classify hand-drawn electronic circuit components was conducted to determine which model is better-suited for the needs of this project. The dataset consisted of 61 classes, including resistors, capacitors (polarized and unpolarized), diodes, transistors (Bipolar Junction Transistor (BJT) and Field-Effect Transistor (FET)), integrated circuits, switches, and more. To ensure a fair comparison, both models were trained and validated on identical data splits.

6.1.2 Performance Metrics

mAP vs Number of Epochs

In order to monitor the progress in models' ability to recognize components as the training progressed, mAP@0.5 was plotted across all epochs. These curves depict how quickly and effectively each model learns over time.

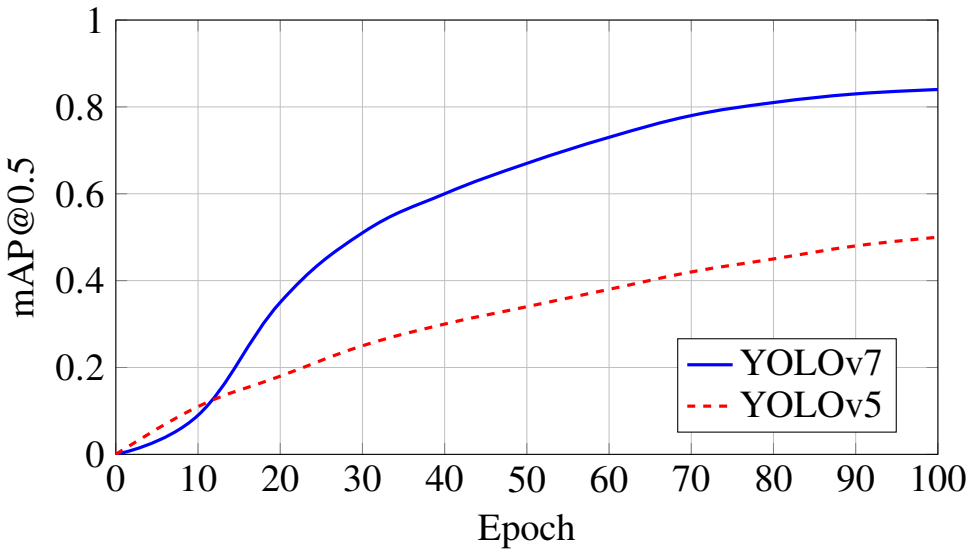
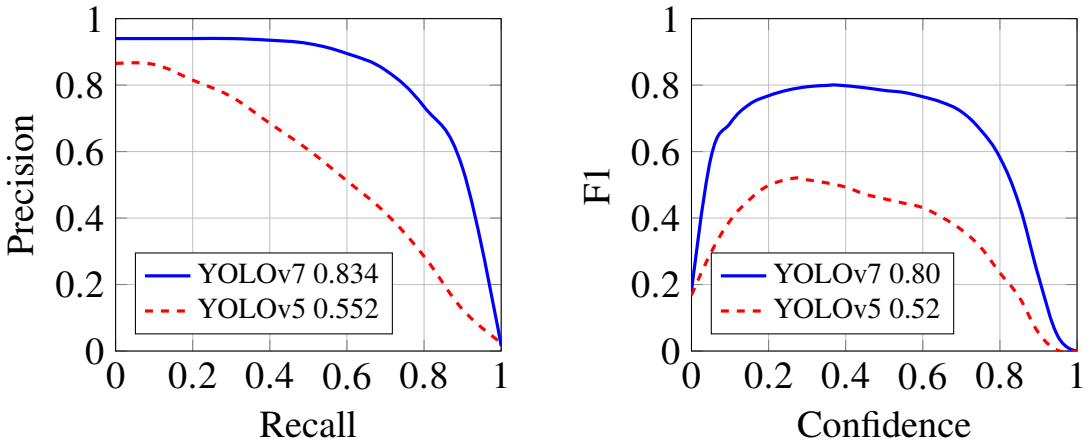


Figure 6-1 mAP@0.5 comparison over 100 training epochs.

It is clear from Figure 6-1 that the mAP score for YOLOv7 are significantly better than that for YOLOv5.

Precision vs Recall and F1 Score vs Confidence

Figure 6-2 demonstrate the F1 Score-Confidence Curve for YOLOv5 and YOLOv7 respectively. The larger the area under this curve, the higher the robustness of the precision-recall trade-off across thresholds.



(a) Precision-Recall curve.

(b) F1-Confidence curve.

Figure 6-2 Performance metrics comparison: (a) Precision-Recall and (b) F1-Confidence curves.

Figure 6-2 also demonstrate the Precision-Recall Curve for YOLOv5 and YOLOv7 respectively. The area under this curve gives the average precision (AP) which informs how good the model is at ranking positive samples above negative ones. The area under each of the curves in these figures is higher for YOLOv7 in comparison to YOLOv5. This is expected behavior and proves YOLOv7 is better than YOLOv5 when it comes to performance.

6.1.3 Quantitative Results Comparison

Final epoch results show YOLOv7 outperformed YOLOv5 across all major metrics.

Table 6-1 Comparison of Performance Metrics for YOLOv5 and YOLOv7

Metric	**YOLOv5**	**YOLOv7**
mAP@0.5	0.61	0.83
mAP@0.5:0.95	0.41	0.60
Precision	0.84	0.93
Recall	0.62	0.78
Epochs Trained	200	100

Overall, YOLOv7 showed a clear advantage over YOLOv5 in our experiments which is why it was picked for component detection in this project.

6.2 Class Taxonomy Consolidation

The CGHD dataset, in its original form, with 61 classes, had classes which looked similar, such as capacitor.polarized and capacitor.unpolarized for instance, and hence were nearly indistinguishable to the component detection model causing frequent inter-class

confusion. To prevent this, visually simular classes were merged under the same descriptor, bringing the number of classes down to 15. This consolidated dataset was used to train four YOLO architectures (YOLOv7, YOLOv8, YOLOv11, YOLOv26) and their results (based on the same performance metrics) were compared with the results amongst themselves as well as with the results from the 61-class dataset.

To keep this comparison fair, the conditions for training on this consolidated dataset (in case of these four YOLO architectures) were kept identical with identical data splits, and confidence and IoU thresholds of 0.25 and 0.45 respectively during evaluation.

6.2.1 Training and Validation Loss: 15-Class Models

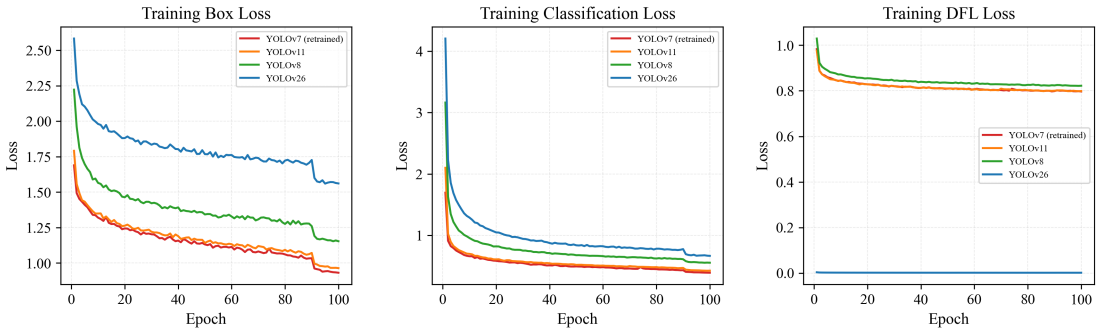


Figure 6-3 Comparison of training loss (four YOLO architectures)

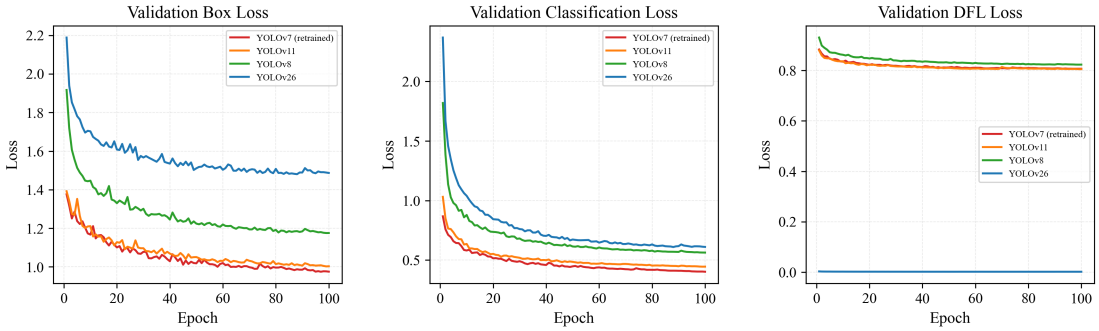


Figure 6-4 Comparison of validation loss (four YOLO architectures)

Figure 6-3 and Figure 6-4 show the training and validation loss for all four models. The training and validation loss for YOLOv7 (retrained) and YOLOv11 declined sharply and reached the lowest value among the four architectures.

6.2.2 Performance Metrics: 15-Class Models

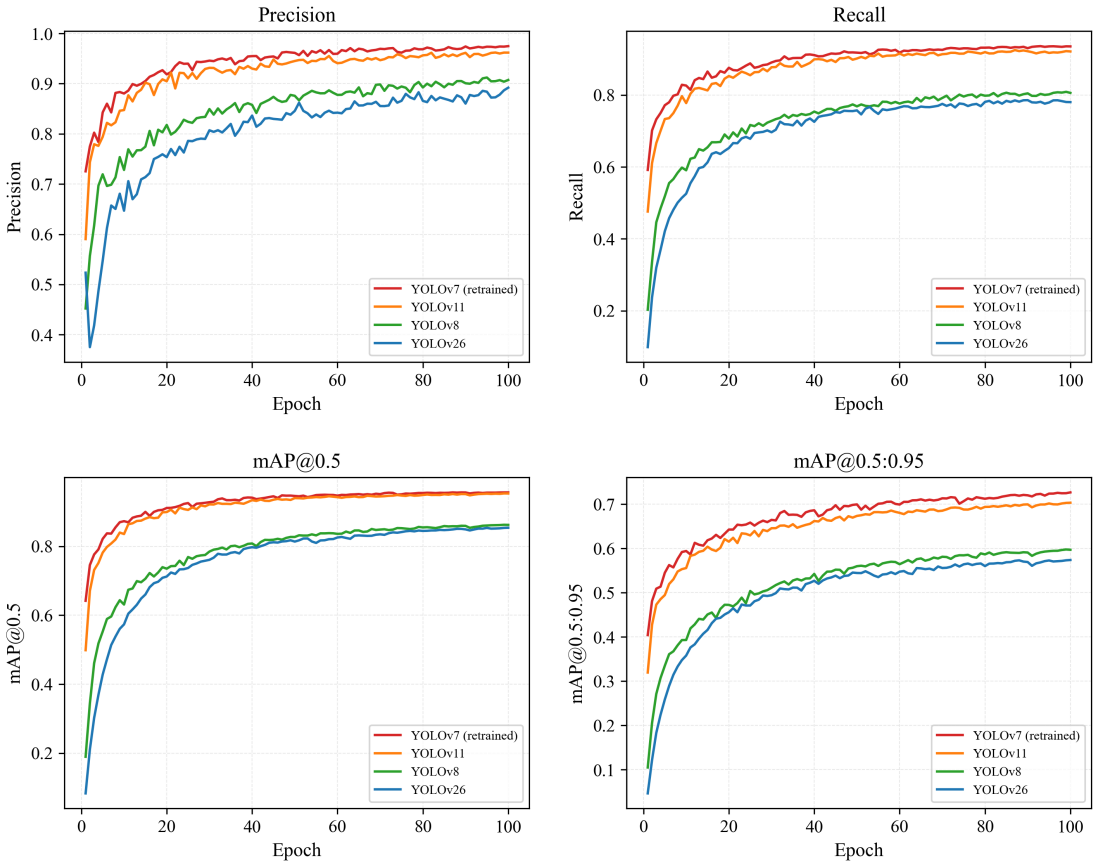


Figure 6-5 Performance metrics vs epochs on the consolidated dataset

Figure 6-5 plots precision, recall, mAP@0.5, and mAP@0.5:0.95 for 100 epochs for all four YOLO architectures. YOLOv7 (retrained) and YOLOv11 plateau at around epoch 40 to 50 reaching their near-maximum values for each performance metric, all of which are consistently higher than the other two YOLO architectures.

6.2.3 Quantitative Results Comparison for 15-Class Dataset

Table 6-2 Final epoch performance comparison of four YOLO architectures on the consolidated dataset

Metric	YOLOv7 (retrained)	YOLOv11	YOLOv8	YOLOv26
Precision	**0.9747**	0.9617	0.9073	0.8919
Recall	**0.9350**	0.9209	0.8058	0.7801
mAP@0.5	**0.9562**	0.9531	0.8620	0.8537
mAP@0.5:0.95	**0.7266**	0.7033	0.5969	0.5738
Epochs	100	100	100	100

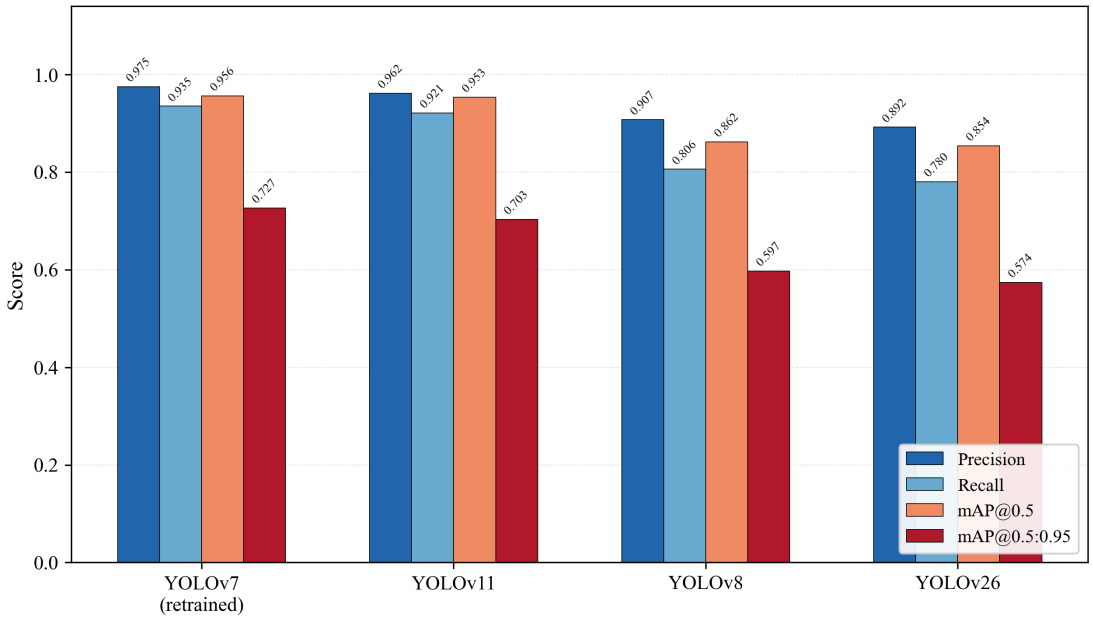


Figure 6-6 Final-epoch performance metrics for four YOLO architectures on the 15-class dataset

Table 6-2 displays the values of performance metrics at the final epoch for all four YOLO architectures. Figure 6-6 displays the same data in a bar chart. YOLOv7 (retrained) is clearly the best models based on all the mentioned metrics, with YOLOv11 following closely. The target value of mAP@0.5 (0.95) is achieved by both of them.

6.2.4 Impact of Class Taxonomy Consolidation

Table 6-3 Effect of class consolidation on YOLOv7 performance

Metric	61-Class	15-Class	Improvement
Precision	0.93	0.9747	+4.8%
Recall	0.78	0.9350	+19.9%
mAP@0.5	0.83	0.9562	+15.2%
mAP@0.5:0.95	0.60	0.7266	+21.1%

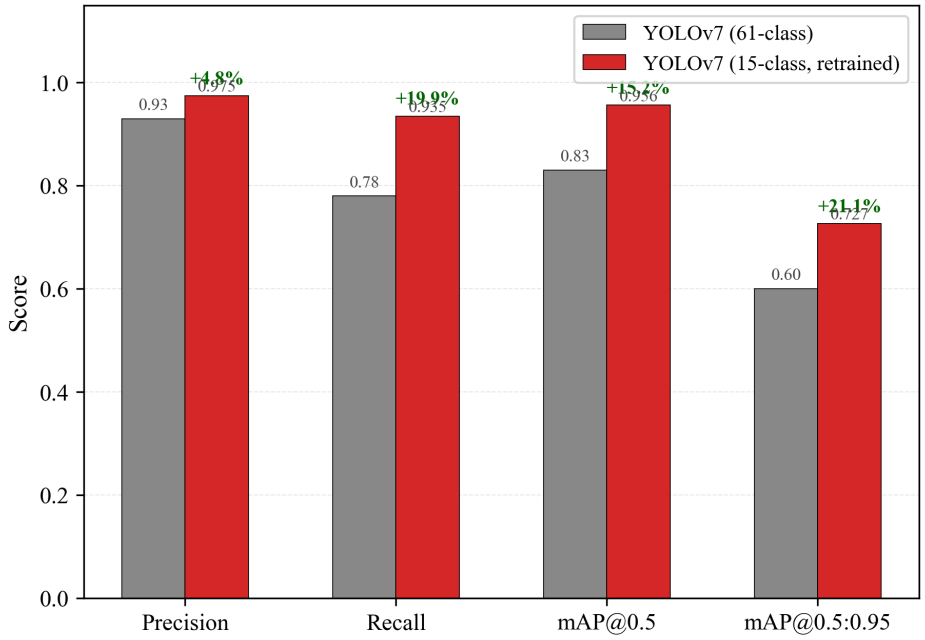


Figure 6-7 Effect of class consolidation on YOLOv7 performance

Figure 6-7 and Table 6-3 show the improvement in performance metrics for YOLOv7 due to consolidation of classes from 61 to 15. This shows that the consolidation, although decreased the nuances in different components, ultimately improved the detection performance.

6.3 Holistic Object Detection Comparison

Table 6-4 Comparison of performance of all object detection models

Model	Classes	Precision	Recall	mAP@0.5	mAP@0.5:0.95
YOLOv5	61	0.84	0.62	0.61	0.41
YOLOv7	61	0.93	0.78	0.83	0.60
YOLOv8	15	0.9073	0.8058	0.8620	0.5969
YOLOv26	15	0.8919	0.7801	0.8537	0.5738
YOLOv11	15	0.9617	0.9209	0.9531	0.7033
YOLOv7 (retrained)	**15**	**0.9747**	**0.9350**	**0.9562**	**0.7266**

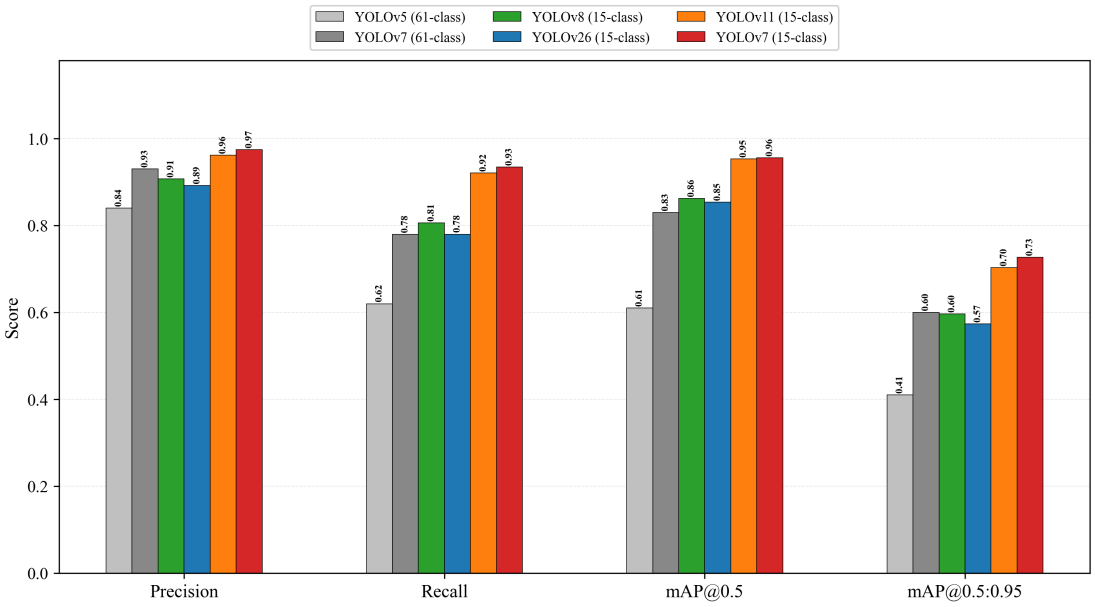


Figure 6-8 Comparison of performance of all object detection models

Table 6-4 and Figure 6-8 respectively tabulate and display the results based on all selected performance metrics for all architectures and class taxonomies.

6.3.1 Radar Visualization of Top Models

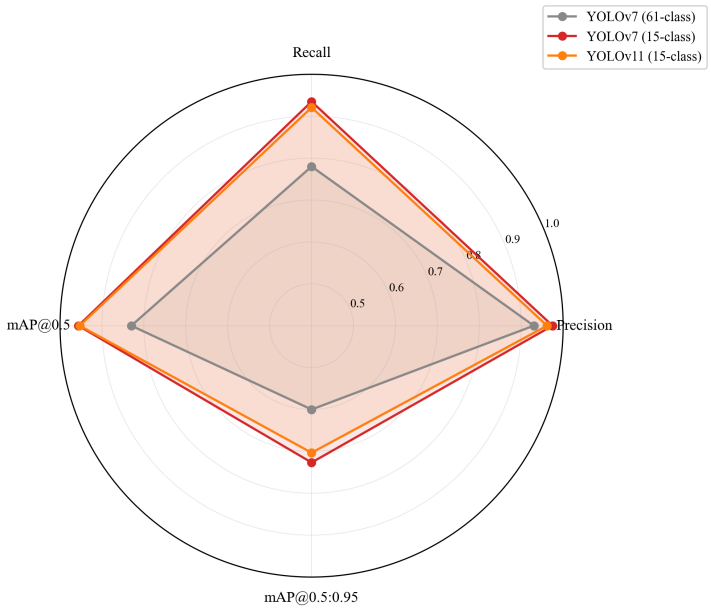


Figure 6-9 Radar chart comparing the three best detection configurations across four performance metrics

Figure 6-9 renders a radar chart of the three strongest configurations. YOLOv7 (61-class) lags visibly in recall (0.78 versus 0.93+), whereas both 15-class models fill out nearly the

entire performance envelope. The retrained YOLOv7 (15-class) marginally outperforms YOLOv11 across all axes, making it the model of choice for the final pipeline.

Overall, from all these findings, it is evident that YOLOv7 (retrained on the 15-class taxonomy) is the optimal object detection model for this project. This also goes to show that class taxonomy might matter more than the architecture at times.

6.3.2 Confusion Matrix for the retrained YOLOv7

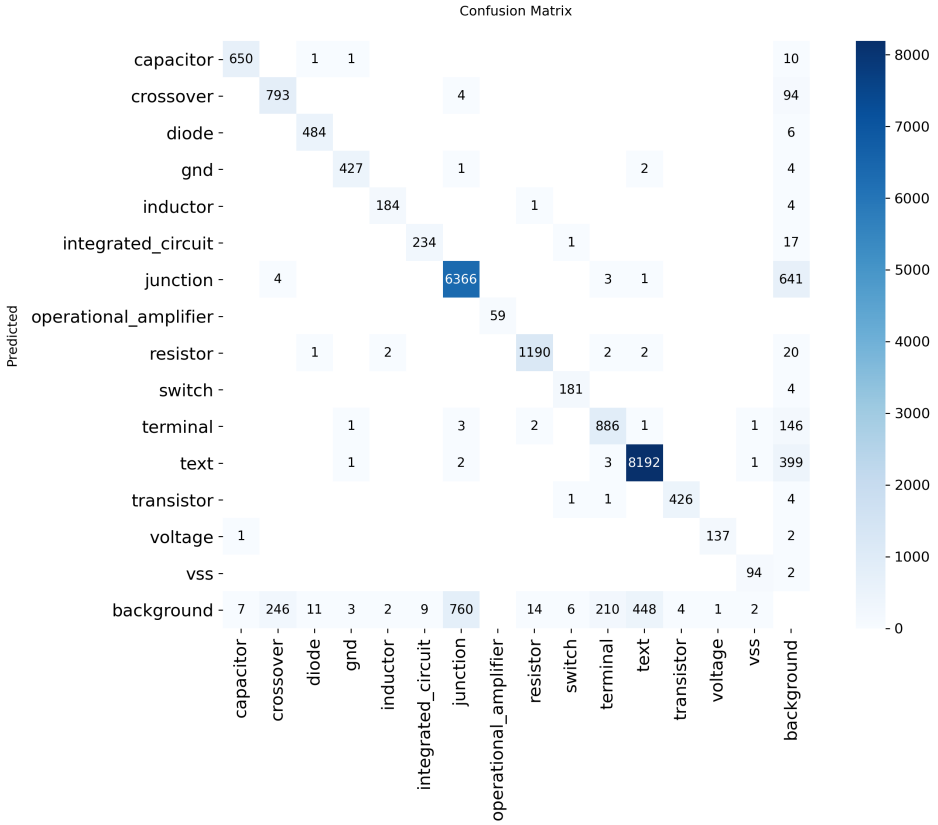


Figure 6-10 Confusion Matrix for the retrained YOLOv7

Figure 6-10 demonstrates the confusion matrix for the retrained YOLOv7 (15 classes). It has a darker main diagonal and very light off-diagonals, which means that the model exhibits strong class separability with minimal inter-class confusion.

6.4 Retrained YOLOv7 Demonstration

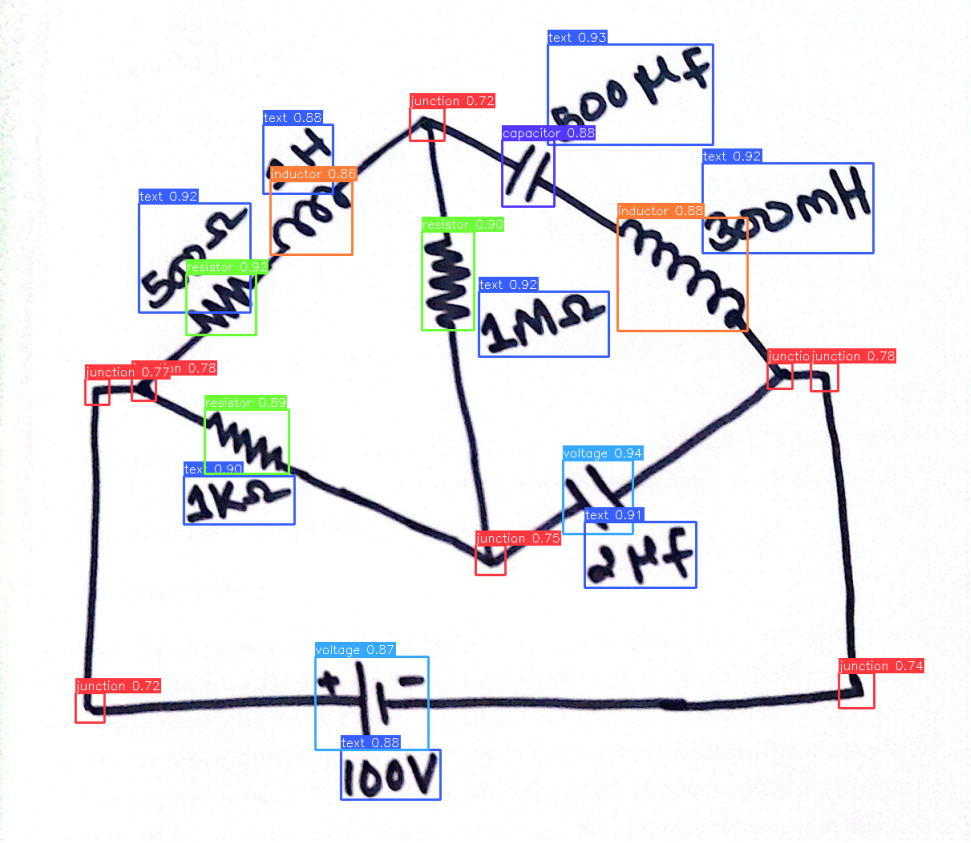


Figure 6-11 Retrained YOLOv7 Result (i)

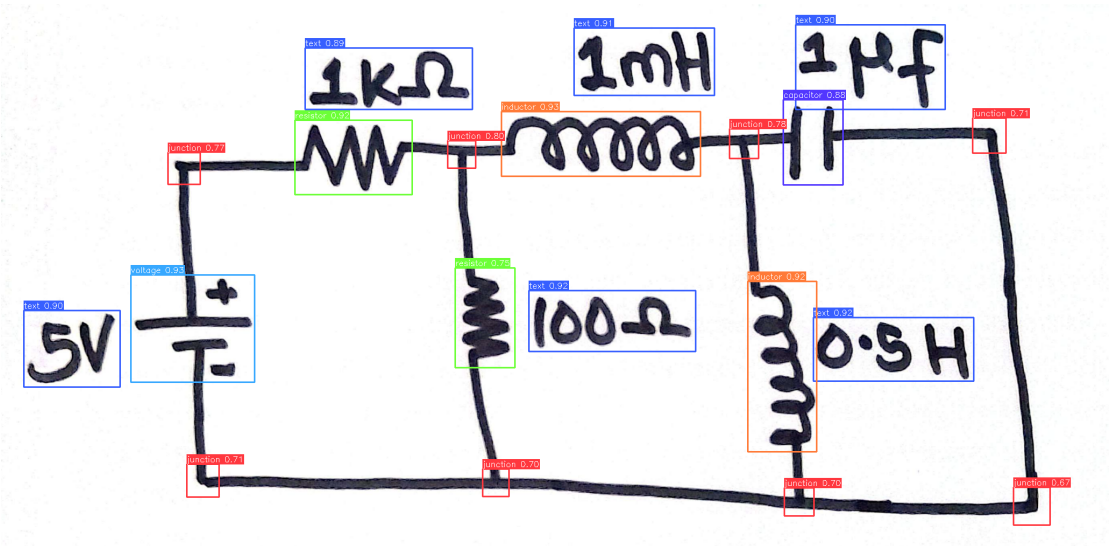


Figure 6-12 Retrained YOLOv7 Result (ii)

Figure 6-11 and Figure 6-12 show the results of the retrained YOLOv7 on two random test images. The model successfully detects and classifies all components in both images

with high confidence scores. These results visually confirm the quantitative metrics and demonstrate the model's effectiveness in real-world scenarios.

6.5 Custom OCR Results and Analysis

6.5.1 Custom OCR Training Loss Curve

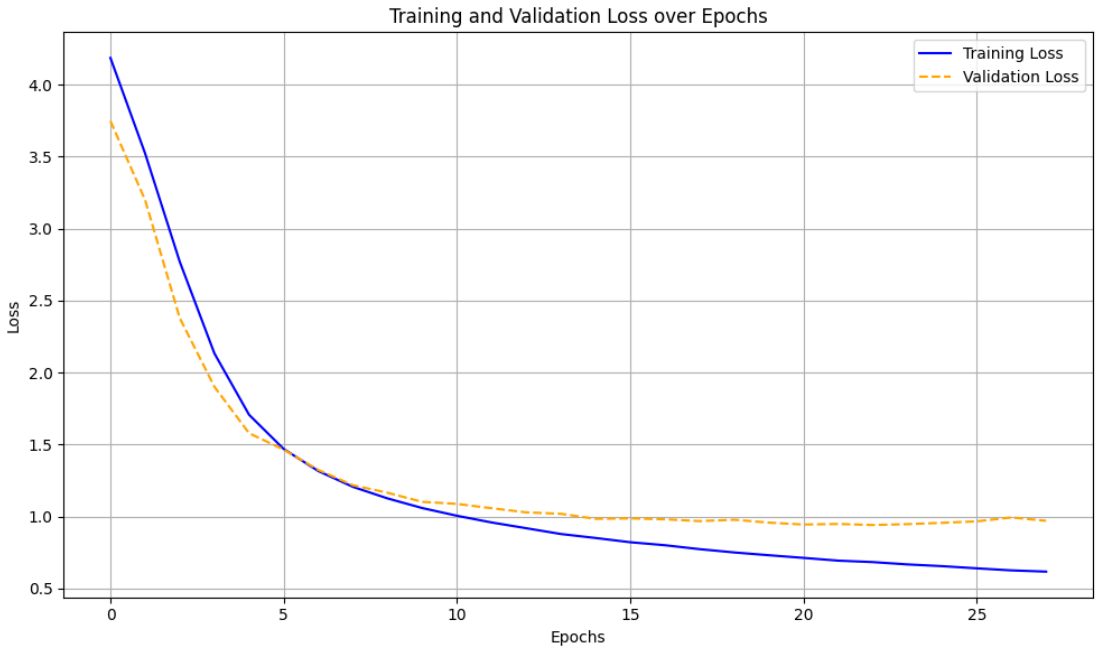


Figure 6-13 Training Loss vs Epochs (Custom OCR)

Figure 6-13 shows that the training loss decreases consistently throughout the 27 epochs, indicating the model is effectively memorizing the training data. There seems to be the slightest uptick in the validation loss, though, at 26 epoch indicating a hint of potential overfitting. The training was stopped at 27 epochs due to early stopping criteria being met.

6.5.2 Custom OCR Performance Analysis

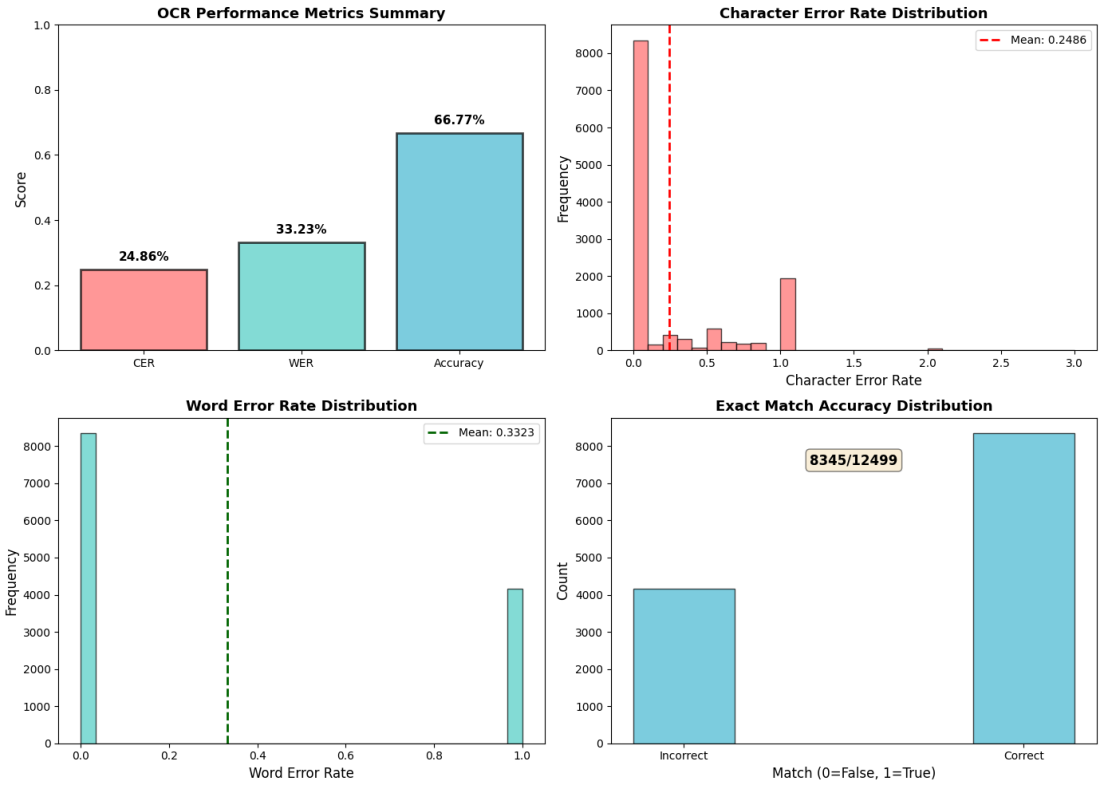


Figure 6-14 Custom OCR Performance Metrics and their Distributions

Figure 6-14 is a combination of images showing the performance metrics for the custom OCR model. The accuracy is 66.77%. Likewise, the CER of approximately 0.25 means that on average, 1 out of every 4 characters is predicted incorrectly. However, the histogram shows a notable bump at 1.0 (complete failure) confirming the presence of "garbage" inputs which can be fixed by the use of data cleaning techniques. WER is shown to be 0.33. Its distribution is inherently binary because the model either gets the full component value right (WER being 0) or gets it wrong (WER = 1).

The performance metrics indicate that while the custom OCR model has learned to recognize some component values, there is significant room for improvement. This sets the stage for TrOCR which is expected to have much better performance due to its pre-trained weights and more advanced architecture.

6.6 Comparative Analysis of OCR Models

Two distinct OCR architectures were evaluated for extracting component values (e.g., “10kΩ”, “100μF”) from detected text regions: the custom CRNN-based model trained from scratch, and a fine-tuned TrOCR-small model based on the Vision Transformer architecture as mentioned in previous sections.

6.6.1 Quantitative Comparison

Table 6-5 Performance comparison of OCR models

Metric	Custom CRNN	Fine-tuned TrOCR
Accuracy (%)	66.77	**84.50**
Character Error Rate (CER)	0.25	**0.12**
WER	0.33	**0.18**
Training Epochs	27	3
Batch Inference	Sequential	Parallel (GPU)
Precision (float)	float32	float16

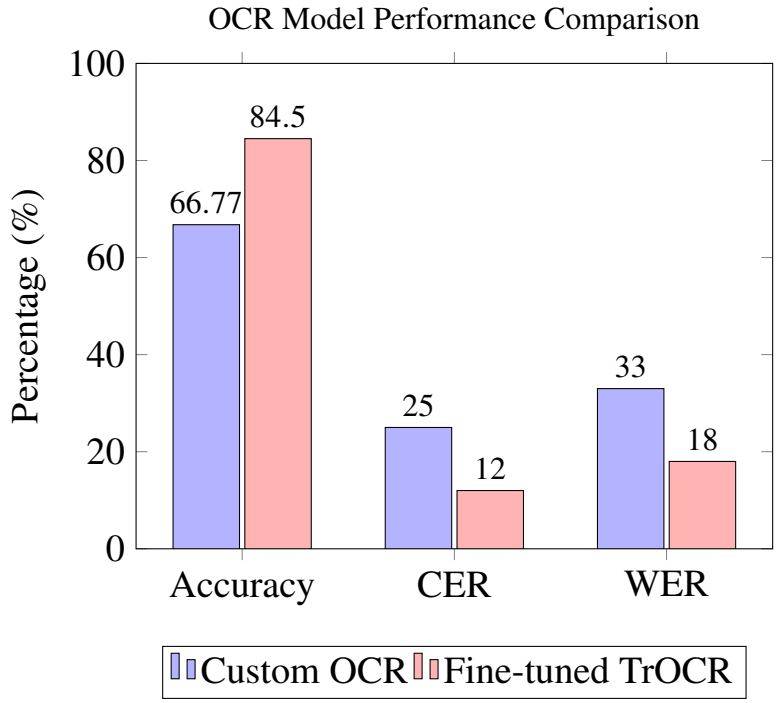


Figure 6-15 Performance metrics comparison, in percentage

Table 6-5 summarizes the performance of both OCR models, and Figure 6-15 provides a visual comparison. The fine-tuned TrOCR model outperforms the custom CRNN across all metrics despite requiring only 3 training epochs compared to 27 (tested on the test split of the CGHD dataset). The CER reduction from 0.25 to 0.12 indicates that TrOCR makes half as many character-level errors, directly improving the accuracy of extracted component values. It is noted that the custom CRNN's lower accuracy (66.77%) was partly attributed to data corruption during preprocessing. Regardless, the architectural advantages of the Transformer-based approach make TrOCR the superior choice for this pipeline. However, the inference speed of TrOCR has found to be considerably slower than the custom OCR model. This implies there is inherently a trade-off between accuracy and inference speed when choosing between the custom OCR and TrOCR

models for this project. Keeping this in mind, both models are retained in the codebase and can be switched based on user preference.

6.7 OCR Demonstration

Two randomly picked images from the dataset are shown below after passing through the detection (YOLOv7) and recognition (OCR) models.

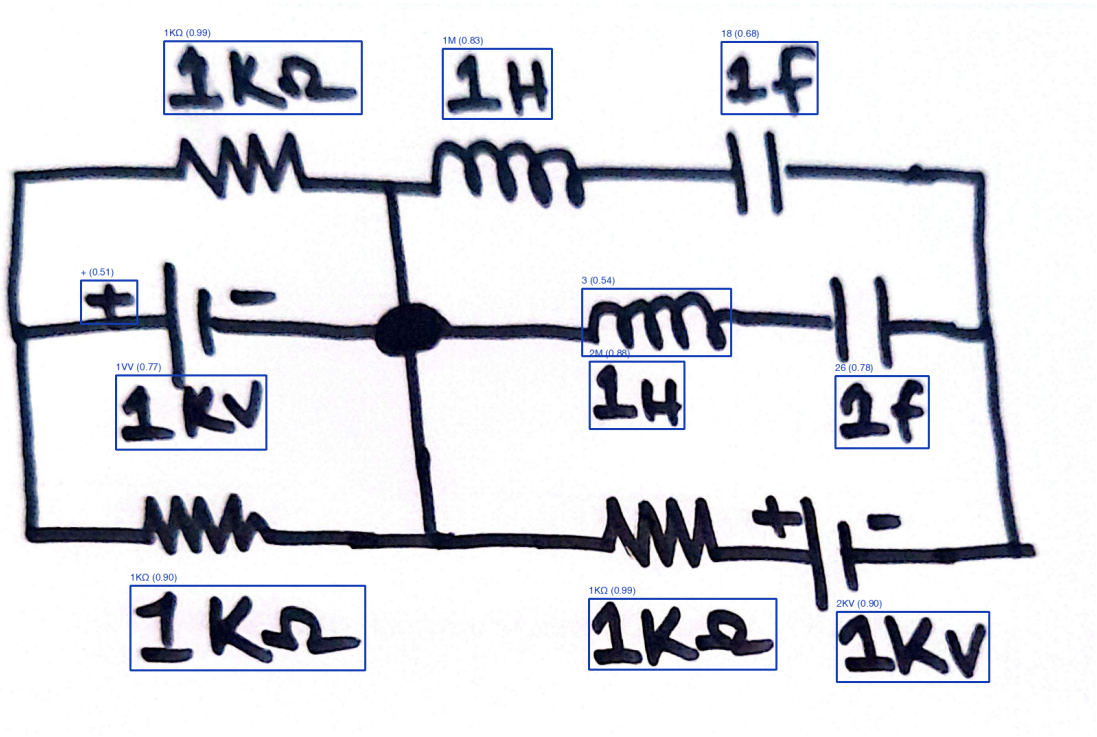


Figure 6-16 Text Detection and Recognition using Custom OCR(i)

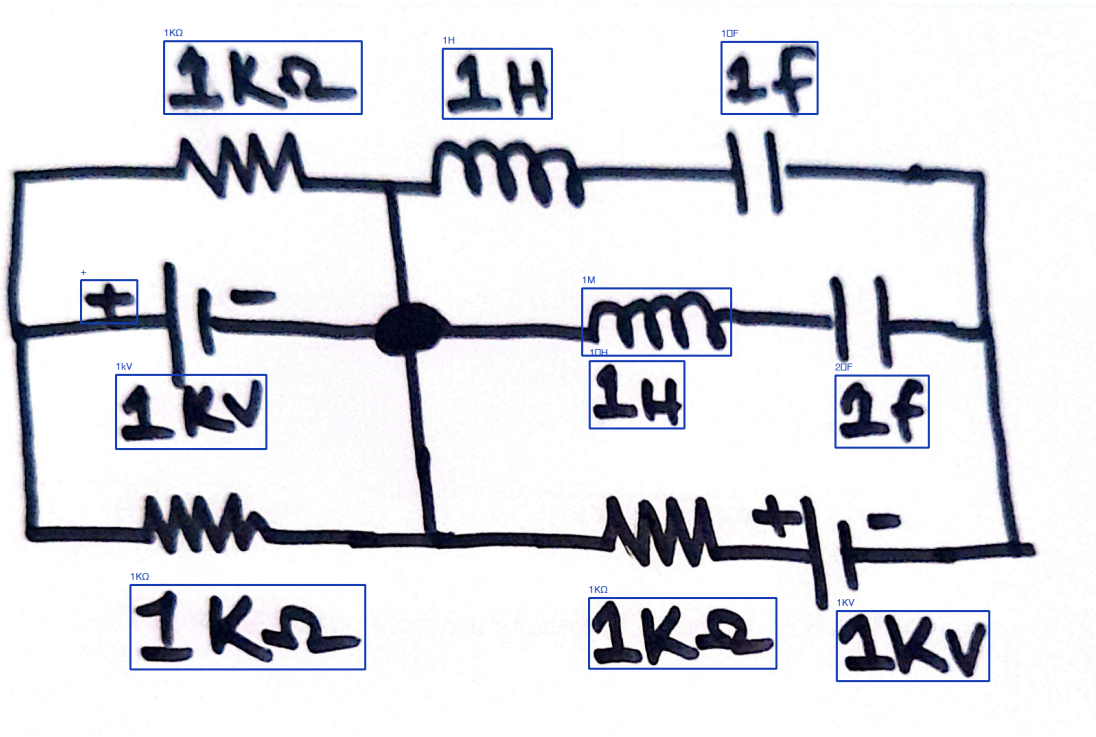


Figure 6-17 Text Detection and Recognition using TrOCR(i)

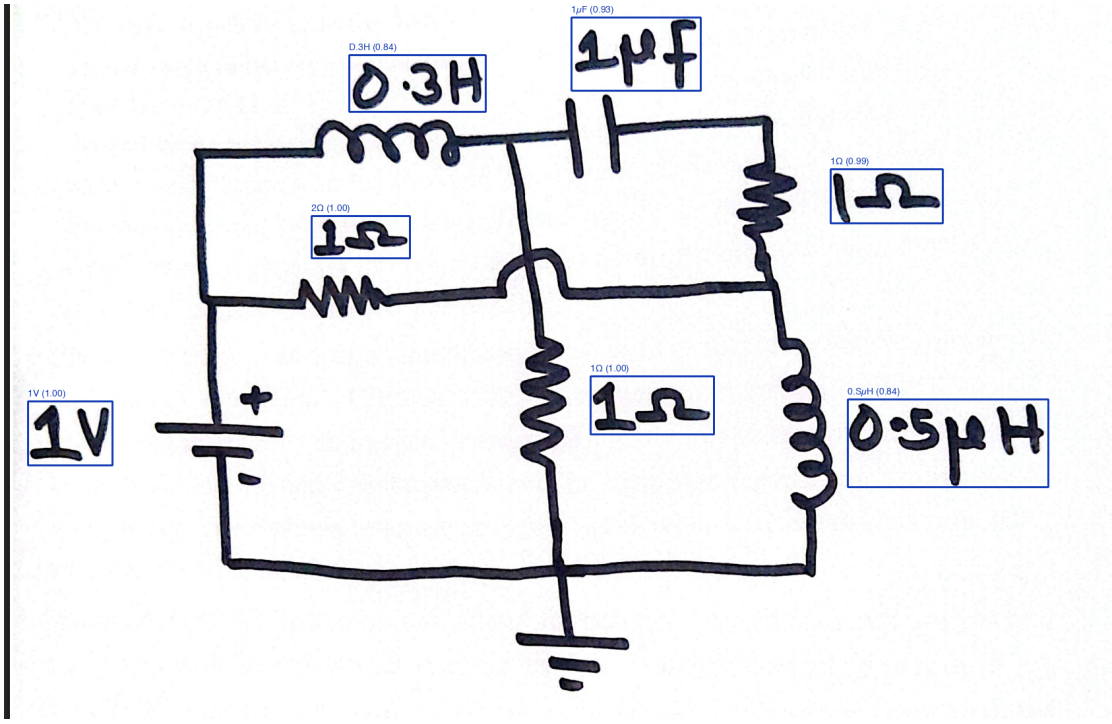


Figure 6-18 Text Detection and Recognition using Custom OCR(ii)

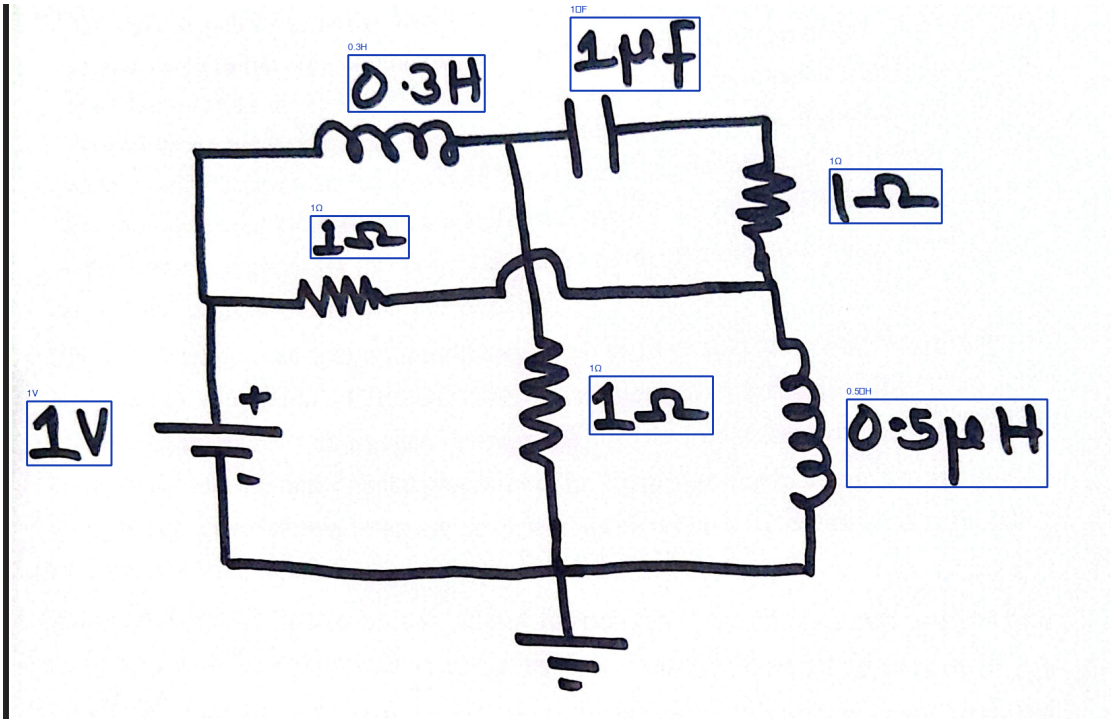


Figure 6-19 Text Detection and Recognition using TrOCR(ii)

Figure 6-16 and Figure 6-18 show the results of the custom OCR model on two random test images. The model successfully detects text regions and extracts component values, albeit with many errors. Figure 6-17 and Figure 6-19 show the results of the fine-tuned TrOCR model which clearly shows better performance recognizing more component values accurately.

6.8 Junction-Based Wire Detection

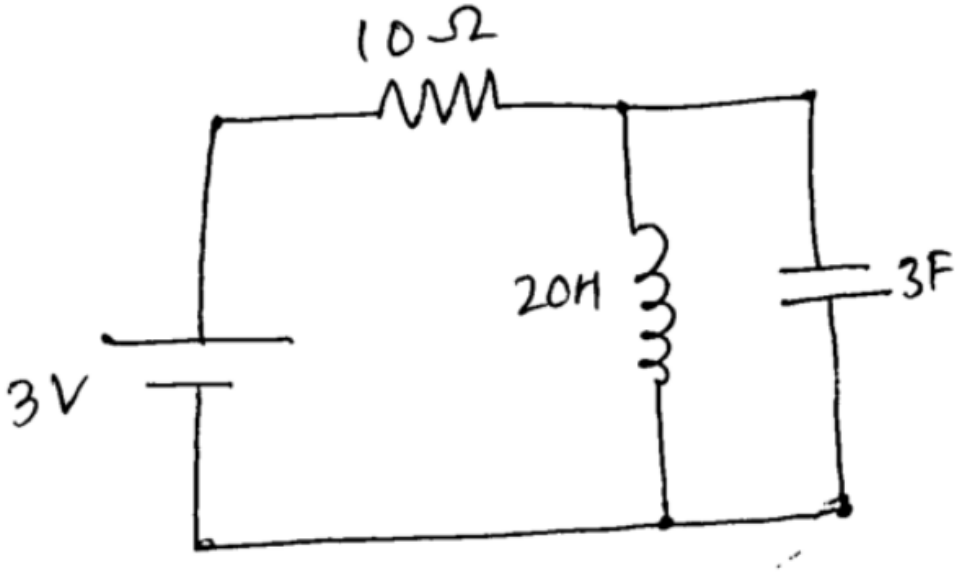


Figure 6-20 Otsu's Global Thresholding

In the junction-based wire detection pipeline, the uploaded image is first binarized using Otsu's Thresholding as shown in Figure 6-20. This binarized image is skeletonized i.e., the width of every line is thinned down to a single pixel and the component bounding boxes are superimposed into this skeletonized circuit as shown in Figure 6-21.

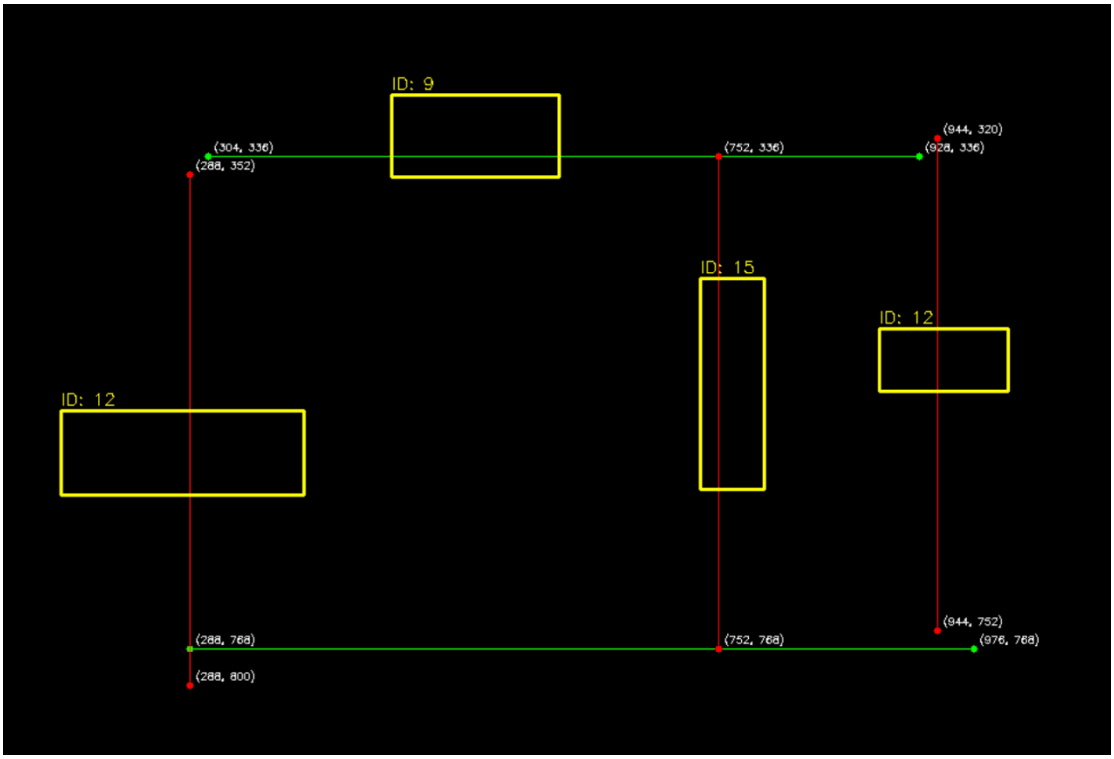


Figure 6-21 Bounding Box Overlay

6.9 Visualization of Line Detection via Path Finding

6.9.1 Skeletonized Image with Detected Endpoints

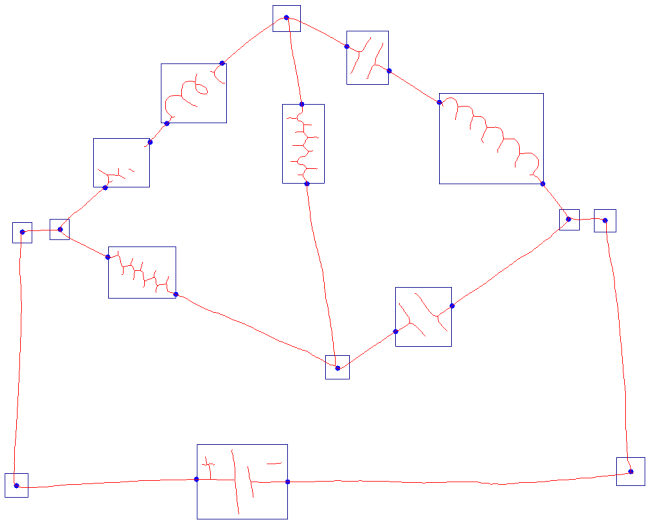


Figure 6-22 Skeletonized Image with Detected Endpoints

Line detection via path finding also requires skeletonization. Figure 6-22 shows the overlay of the bounding boxes in the skeletonized circuit. The blue rectangles signify the bounding boxes as given by the YOLO model, and the blue dots signify the endpoints (used as nodes for BFS).

6.9.2 Adjacency Graph Visualization

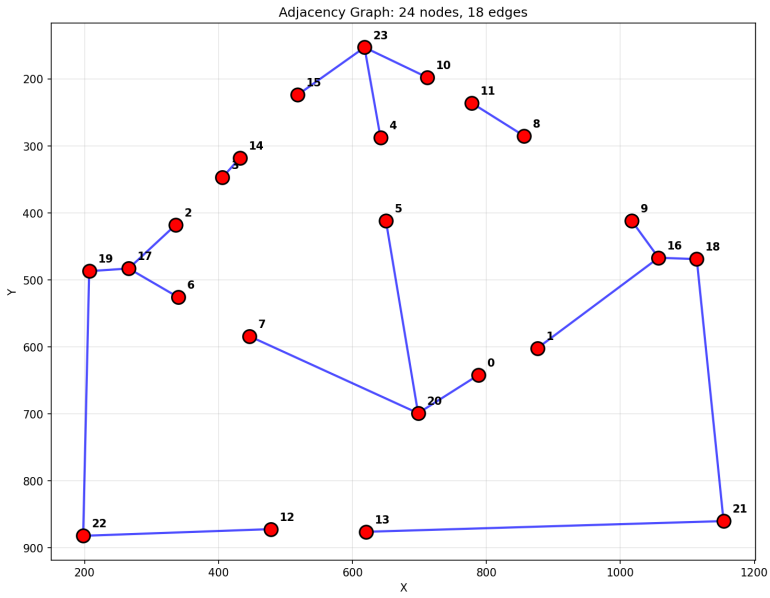


Figure 6-23 Adjacency graph constructed using Multi-Head BFS algorithm

Figure 6-23 shows the adjacency graph produced by the Multi-Head BFS algorithm with blue lines representing the edges between the red nodes (the self edges have been pruned). The empty spaces between some nodes are where the circuit components were situated in the sketch and will later be added in the digital schematic.

6.10 Combined Overlay of YOLOv7, TrOCR, and Line Detection Results

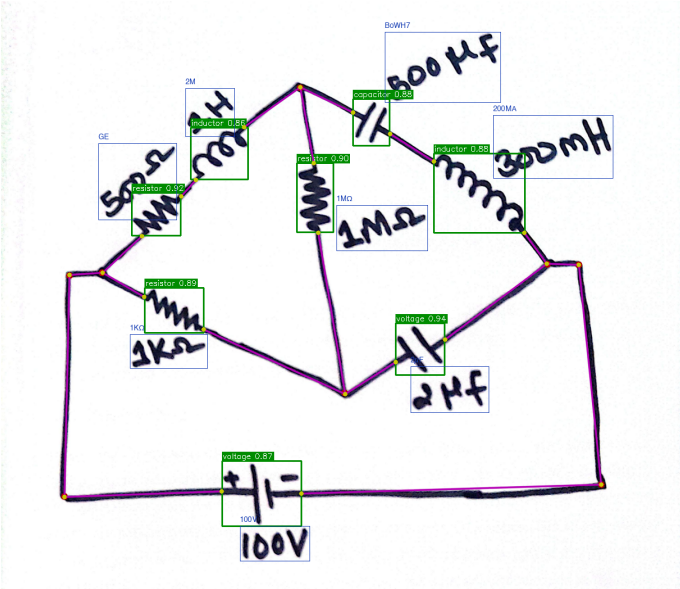


Figure 6-24 Combined Results of YOLOv7, TrOCR, and Line Detection

Figure 6-24 shows the combined effect of the result from component detection (green bounding boxes), text recognition (blue rectangles with text annotations), and line detection (red wire tracings) overlaid in a Whetstone bridge circuit.

6.11 Frontend Visualization

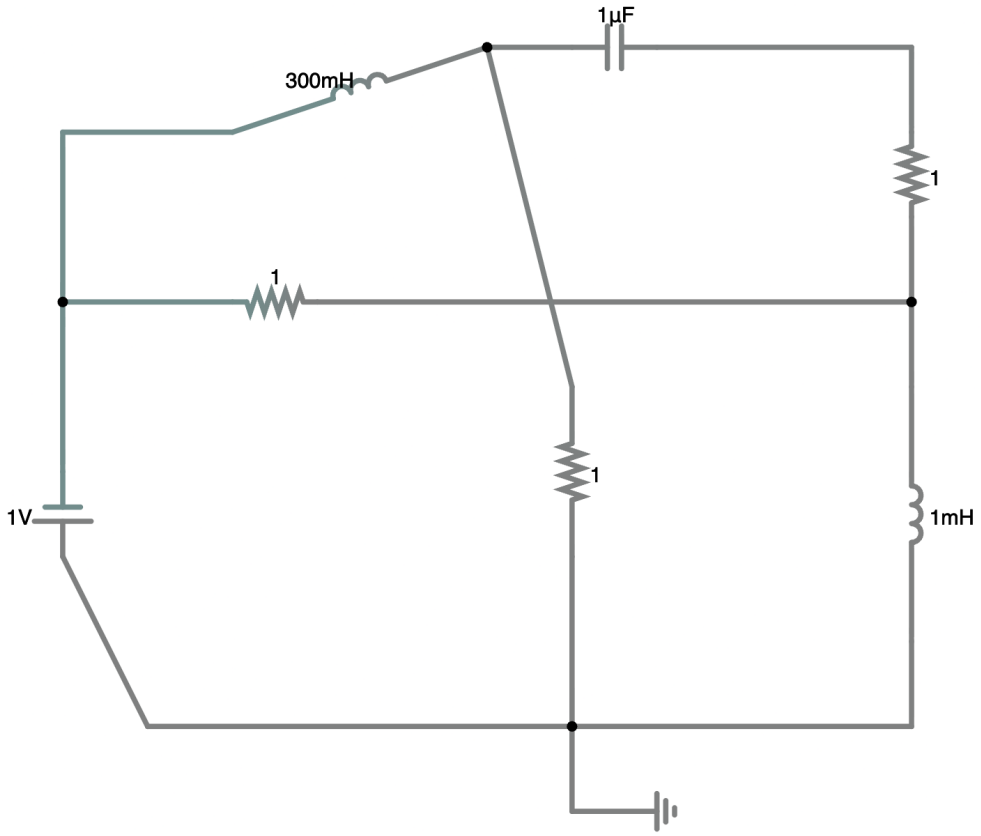


Figure 6-25 Digitally Drawn Circuit (Color Inverted)

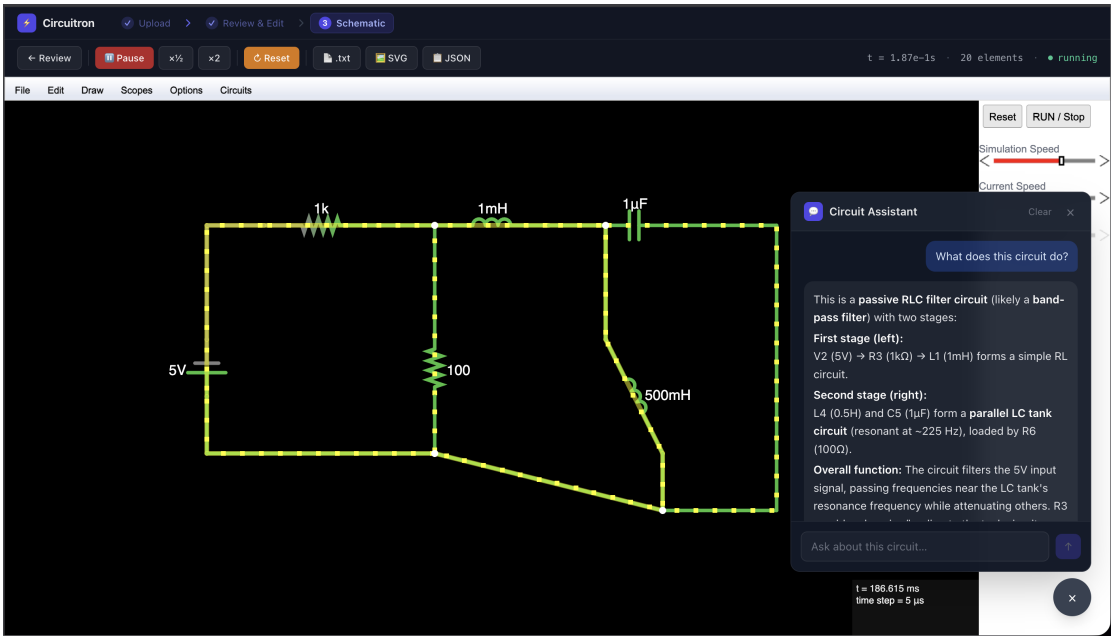


Figure 6-26 Fullscreen Capture of the Frontend

Figure 6-25 shows the digitized version of the hand-drawn circuit fed into the system. This is just the automatically generated schematic before any manual intervention. Figure 6-26 shows the full-screen capture of the frontend of the system with a circuit diagram, options to modify this diagram in the menu bar, simulation setting towards the right, and also the LLM-powered Q&A feature. Measurement probes let users monitor signals in real time while the system automatically computes peak-to-peak swing and Root Mean Square (RMS) values alongside maxima and minima. Simulation data and plots can also be exported in standard formats for offline review or inclusion in documentation.

7 FUTURE ENHANCEMENTS

The CIRCUITRON system, while fully functional in its current state, has numerous opportunities for future enhancements. One of the objectives of any research-based project is to encourage the scientific community to conduct further studies and experiment on the working principle of the project to either verify the methodology used, or potentially come up with alternate techniques to solve the same probelem. In any case, following are the likely future enhancements that can be implemented in this project:

7.1 Improvement in the OCR accuracy

The system uses two OCR models, at the moment, however the accuracy of both of them (highest of them being around 85%) has room for improvement. Also, the issue with TrOCR (which is the better of the two) is that it is too slow on account of its large size. The custom OCR pipeline is much faster, but has lower accuracy. Future work can be done to either improve the accuracy of the custom OCR model though rigorous training on different datasets, or to somehow improve TrOCR's speed.

7.2 Increment in the number of supported components

The system currently supports a limited number of components (only 12 after consolidation, minus crossover, junction, and text classes). Pre-consolidation, the system supported 59 components, but the mAP was too low for satisfactory object detection. These 12 components are the most commonly used ones for basic circuit design and simulation, however, there are many more components that can be added to the system to make it more versatile and useful for a wider range of applications. Future work can be done to increase the number of supported components, which would require training the object detection model on a larger and more diverse dataset.

7.3 Authentication and Database Functionalities

The system, in its current form, doesn't allow the user to save their work or to access it later. Moreover, there is no authentication mechanism in place to allow users to have personalized accounts and potentially collaborate on the same projects. This also involves hosting the system online. Future enhancements can include the integration of a database to store user data and circuit designs, as well as offer a library of pre-designed circuits for beginners to play around with.

7.4 Improvement in the frontened and general user experience

While the system is capable of handling diagonal wires and components, when said components are rendered in the frontend, they aren't rendered in the same diagonal

manner. This requires the user to step in and manually adjust the orientation of the component. The issue of scale, between the wires and the components also persists, ocassionally requiring manual resizing. Moreover, the frontend of the system involves a lot of user interactions from dragging and resizing, to general drawing and editing of the schematic, which can be made smoother in the future.

8 CONCLUSION

The objectives of recognizing a hand-drawn circuit diagram, transforming it into an editable schematic having simulation capabilities, and analyzing said circuit with LLM-powered Q&A have been successfully achieved. Functional pipelines for each aspect of the project have been designed and implemented effectively. For component detection, YOLOv7 has achieved a mAP@0.5 of 95% and for text recognition, TrOCR has achieved accuracy of 84.5%. Through the use of junction-based geometric inference and collinear merging, the wire detection algorithm extracts circuit connectivity. Overall, all these elements work hand-in-hand to extract components with their values and connections, effectively digitizing a circuit. Hence, the project has successfully addressed a significant gap in educational circuit analysis by making circuit simulation easier than ever.

Beyond the ML inference side of things, the project also pulls off a working full-stack deployment: a React.js frontend talks to a FastAPI backend, which in turn hooks into CircuitJS for running simulations.

CIRCUITRON also doubled as a research exercise for the authors. Multiple approaches for solving the same problem were tried out (different YOLO models for component detection, custom OCR and TrOCR for text recognition, and junction-based and path finding algorithms for wire connectivity) and the better alternative was selected in each case with limitations of the discarded method explained.

9 APPENDICES

Appendix A: Project Budget

Table 9-1 Project Budget Breakdown

Category	**Details**	**Estimated Cost (NPR)**
Computation Requirements	Google Colab Pro ($9.99 per month), NVIDIA T4 GPU ($0.35 per hour), Lightning AI Credits	24,000
IEEE Xplore Membership	Annual subscription for accessing research articles, journals	2,160
Miscellaneous Costs	Report printing, binding, API, etc.	15,000
Total		**41,160**

Appendix B: Gantt Chart

Table 9-2 Project Timeline (Gantt Chart)

PROCESS	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar
Research											
Implementation											
Testing											
Documentation											
Final Submission											

REFERENCES

[1] R. R. Reddy and M. R. Panicker, "Hand-drawn electrical circuit recognition using object detection and node recognition," *arXiv preprint arXiv:2106.11559*, Jan 2021, available: https://arxiv.org/abs/2106.11559.

[2] A. E. Sertdemir, M. Besenk, T. Dalyan, Y. D. Gokdel, and E. Afacan, "From image to simulation: An ANN-based automatic circuit netlist generator (Img2Sim)," in *Proc. IEEE Int. Conf. on Synthesis, Modeling, Analysis and Simulation Methods and Applications to Circuit Design (SMACD)*, 2022, pp. 1–4.

[3] Y.-L. Chen, Y.-R. Hong, Y.-T. Sung, and K.-E. Chang, "Efficacy of simulation-based learning of electronics using visualization and manipulation," *Educational Technology & Society*, vol. 14, no. 2, pp. 269–277, 2011.

[4] M. F. Adnan, N. Ahmed, A. Shafiullah, M. S. Rahman, and G. Sarowar, "A two-stage CNN-based hand-drawn electrical and electronic circuit component recognition system," *Soft Computing*, vol. 33, pp. 13 367–13 390, 2021.

[5] B. Edwards and V. Chandran, "Machine recognition of hand-drawn circuit diagrams," in *Proc. IEEE Int. Conf. on Acoustics, Speech, and Signal Processing (ICASSP)*, vol. 6, Nov 2000, pp. 3618–3621.

[6] C. R. Kelly and J. M. Cole, "Digitizing images of electrical-circuit schematics," *APL Machine Learning*, vol. 2, no. 1, p. 016109, Jan 2024.

[7] Y. Yamakaji, H. Shouno, and K. Fukushima, "Circuit2graph: Circuits with graph neural networks," *IEEE Access*, vol. 11, pp. 123 456–123 467, 2023.

[8] J. Bayer, A. K. Roy, and A. Dengel, "Instance segmentation based graph extraction for handwritten circuit diagram images," *arXiv preprint arXiv:2301.03155*, Jan 2023, available: https://arxiv.org/abs/2301.03155.

[9] M. S. Abhinav, V. L. Shrimanth, P. N. Ganesh, P. V. Nishanth, and K. Bhargavi, "Electric circuit diagram detection, recognition and simulation," *International Research Journal of Engineering and Technology (IRJET)*, vol. 7, no. 9, pp. 3221–3227, Sep 2020.

[10] D. Hemker, J. Maalouly, H. Mathis, R. Klos, and E. Ravanan, "From schematics to netlists–electrical circuit analysis using deep-learning methods," *Advances in Radio Science*, vol. 22, pp. 61–75, Nov 2024.

[11] KiCad, "SPICE simulation," https://www.kicad.org/discover/spice/, 2024, accessed: 2025-01-15.

[12] D. Vungarala, S. Alam, A. Ghosh, and S. Angizi, "SPICEPilot: Navigating SPICE code generation and simulation with AI guidance," *arXiv preprint arXiv:2410.20553*, Oct 2024, available: https://arxiv.org/abs/2410.20553.

[13] P. Falstad, "Circuit simulator," https://www.falstad.com/circuit/circuitjs.html, 2026, accessed: Mar. 22, 2026.

[14] J. Solawetz, "What is YOLOv7? a complete guide," https://blog.roboflow.com/yolov7-breakdown/, 2024, accessed: 2026-05-13.

[15] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: Transformer-based optical character recognition with pre-trained models," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 13 094–13 102, arXiv:2109.10282.